

Expense Management System

Manage and submit ERR Expenses efficiently with our secure web application.

March 2026

ERR!!!: Full Stack Expenses Web App with ROS API Integration for Enhanced Reporting Requirements (ERR)

Project Report

Name:	Elaine Rice
Student ID:	20108879
Programme:	Higher Diploma in Computer Science (NFQ Level 8)
Institution:	SETU Waterford – Department of Computing & Mathematics
Module:	Project (ICT Placement Skills)
Project Supervisor:	Sonya Hogan (Sonya.Hogan@setu.ie)

Declaration of Authenticity

I declare that the work presented in this report is my own, and that any quotations from any sources (e.g. books, journals, the internet) are clearly identified as such. All quotations and paraphrases are accompanied by in-text citations (author, year) and a full reference in the bibliography. I confirm that I have not submitted the work represented in this report in any other course of study leading to an academic award.

Student: ____ Elaine Rice _____ Date: ____ 21/03/2026 _____

Acknowledgements

I acknowledged the guidance and feedback provided by my project supervisor Sonya Hogan, my workplace colleagues, for their support in the planning, implementation, and evaluation of this project. I also acknowledged the maintainers of the open-source tools and documentation used throughout development. My partner, who supported me wholeheartedly over the last two years while completing the course.

Version History			
Version	Description	Author	Date
1.0	Interim Report	Elaine Rice	15/02/2026
2.0	Final Report	Elaine Rice	21/03/2026

Table of Contents

Declaration of Authenticity	1
Acknowledgements	1
1. Abstract	6
2. Introduction.....	6
2.1 Problem Statement	6
2.2 Project Aim	6
2.3 Scope	6
2.4 Deliverables	7
2.5 Acronyms.....	7
3. Research and Analysis	7
3.1 Research	7
3.2 Analysis	8
3.3 Existing Solutions (Internal and External).....	8
3.4 POC Functional Requirements.....	9
3.5 Technology Choices	9
3.6 ROS API Integration Considerations	9
4. Methodology and Project Planning	10
4.1 Agile Scrum Methodology	10
4.2 Planning and Tracking (Trello)	10
5. System Design.....	11
5.1 Architecture Model	11
5.2 Data Model	12
5.3 UI Designs and Workflow	13

5.3.1 Home Screen (/home)	15
5.3.2 About hyperlink (/about).....	15
5.3.3 Log in Screen (/login).....	16
5.3.4 Initial log in for the Processor Role (/processor).....	16
5.3.5 Employee Management (/processor/employees)	18
5.3.6 Element Management (/processor/elements).....	20
5.3.7 Company Management (/processor/company)	21
5.3.8 Submissions (/processor/submissions)	22
5.3.9 Processor Reports (/processor/reports).....	24
5.3.10 Initial log in for the Admin Role (/admin).....	25
5.3.11 Redirect to Processing Functionality (/processor (admin role)).....	25
5.3.12 Revenue Integration (/admin/revenue)	26
5.3.13 Admin Reports (/admin/reports)	29
5.3.14 Logout Menu Option (logout() → /login)	30
5.4 Security and Privacy by Design	30
6. Implementation	31
6.1 Backend Architecture and Security (see Appendix A diagram)	31
6.2 Frontend Architecture and User Interface	31
6.3 Claims Processing and Business Logic	32
6.4 ROS API Integration Layer.....	32
6.5 ERR Submission Schema Design	32
6.6 Submission Modelling Integration.....	32
6.7 ERR Submission Generation Engine	33
6.8 Data Normalisation and End-to-End Validation	33
6.9 Conditional Revenue Field Handling.....	33
6.10 Design of the .NET Signing Microservice	33
6.11 Revenue ROS Certificate Cryptographic Signing Engine Implemented.....	34
6.12 Revenue PIT3 handshake fully operational using .NET signing microservice.....	34
6.13 ERR Submission Lifecycle and ROS API Integration	34
6.14 ERR Submission Lifecycle from UI.....	38
6.15 Employee Management from UI	39
6.16 Element Management from UI.....	40

6.17 Company Configuration from UI	40
6.18 System Reporting and Governance Layer	41
6.19 Edit Functionality for Employees, Elements and Claims (CRUD Completion)	42
7. Testing and Validation	43
7.1 Test Strategy	43
7.2 Sample Test Cases	43
8. Results and Evaluation.....	43
9. Critical Self-Review, Reflection and Future Improvements.....	44
9.1 What Was Achieved	44
9.2 What Was Learned	44
9.3 Problems Encountered and Solutions	44
9.3.1 Certificate Handling Limitations in Node.js	44
9.3.2 Password Handling Discrepancies	44
9.3.3 OpenSSL Extraction Attempts.....	45
9.3.4 Architectural Misalignment Between Node.js and Revenue PIT3.....	45
9.3.5 Decision to Introduce a Minimal .NET Signing Microservice.....	46
9.4 What I Would Do Differently	46
9.5 Future Improvements.....	46
10. Use of Artificial Intelligence (Appendix C)	46
11. Conclusion	47
References	48
Appendices	50

Table of Figures

Figure 1: Agile Sprint Planning.....	10
Figure 2: Architecture Model	11
Figure 3: Database Collection	12
Figure 4: System User Flow	14
Figure 5: UI Landing Page	15
Figure 6: About Hyperlink.....	15
Figure 7: About URL.....	15
Figure 8: Login Screen	16
Figure 9: Back to Home Button	16

Figure 10: Expense Claims Menu Button (displaying logged in user).....	16
Figure 11: Processing Claims Dashboard.....	17
Figure 12: Select Claim for Editing.....	17
Figure 13: Editing Pending Claim.....	17
Figure 14: Employee Management Dashboard.....	18
Figure 15: Select Employee for Viewing / Editing	18
Figure 16: Edit Employee Option.....	19
Figure 17: Editing Employee Record.....	19
Figure 18: Elements Management Dashboard.....	20
Figure 19: Select Element for Editing	20
Figure 20: Edit Element Option	21
Figure 21: Company Details Dashboard	21
Figure 22: Submission Dashboard	22
Figure 23: Display Pending Submission Line Items.....	22
Figure 24: Send Submission Button.....	22
Figure 25: Submission Acknowledgement: "Submitted"	23
Figure 26: Submission Acknowledgement: "Failed"	23
Figure 27: Processing Reporting Dashboard	24
Figure 28: User Management Dashboard	25
Figure 29: Admin Redirect to Processing Functionality.....	25
Figure 30: Dynamic Admin Menu Button.....	25
Figure 31: Revenue Integration Dashboard	26
Figure 32: Successful ROS Handshake	26
Figure 33: Cert Issues; Revenues Confirmation.....	27
Figure 34: Unsuccessful ROS Handshake.....	27
Figure 35: Signing Service Health Check Output	27
Figure 36: ROS Cert Metadata and Base64 Encoding Output	28
Figure 37: ROS Certificate RSA-SHA512 Signing Output.....	28
Figure 38: Admin Reports Dashboard; Claims Activity.....	29
Figure 39: Admin Reports Dashboard; Submission Activity	29
Figure 40: Logout Button.....	30
Figure 41: Logout Redirect to Login Screen.....	30
Figure 42: Successful ROS Submission Acknowledgement	37
Figure 43: System Architecture Diagram: High-Level.....	50
Figure 44: Architecture Diagram: Client Layer	51
Figure 45: Architecture Diagram: Application Layer	51
Figure 46: Architecture Diagram: Data Layer	52
Figure 47: Architecture Diagram: Cryptographic Service Layer	53
Figure 48: Architecture Diagram: ROS Integration Layer	53

1. Abstract

This project delivers a proof-of-concept (POC) full-stack Expenses web application that captures, manages, and prepares employee expense records for Enhanced Reporting Requirements (ERR) submission to Revenue.

In Ireland, ERR reporting is mandated under Section 897C of the Finance Act 2022 (Finance Act 2022, s.897C; Revenue, 2024). These rules require employers to report non-taxable benefits and expenses such as travel, subsistence, and the small benefit exemption, to Revenue on or before the payment date via the Revenue Online Service (ROS).

This POC project aims to integrate with ROS API to demonstrate an end-to-end ERR submission workflow. The application is designed as a standalone SaaS-style web product. The implemented solution provides user authentication, RBAC and secure storage of data, while operating entirely on a dummy dataset to avoid processing personal or sensitive information. The project followed an iterative agile approach with short sprints.

2. Introduction

This report documents the design and implementation of the POC web application. The project was completed as a work-based project and targeted a transition from a desktop application to a modern cloud based SaaS web application.

2.1 Problem Statement

The organisation's payroll desktop application, MegaPay, supports ERR submissions. However, feedback indicated that the workflow was difficult to navigate and required multiple steps that were not aligned with modern user expectations. This project produced a standalone web application that simplified the ERR process while maintaining the essential reporting workflow and statutory compliance. This aligned with the organisations strategic shift from on-premise software to a cloud-based SaaS offerings.

2.2 Project Aim

The project aimed to demonstrate that a web application could capture and manage expense data and integrate directly with the ROS API to submit ERR compliant payloads as part of an auditable submission workflow.

2.3 Scope

The scope was a POC using a dummy dataset. No live personal data was processed. The system implemented core workflows: user authentication, RBAC, expense capture, categorisation, ERR payload preparation, ROS API submission integration, and submission reporting. Non-functional requirements such as encryption were incorporated as design decisions and partially implemented where appropriate for a POC (European Union, 2016).

2.4 Deliverables

The project deliverables include:

- A full-stack web application prototype with a ROS API integration layer to submit ERR payload submission and capture responses.
- A final project report documenting: models, analysis, design, iteration plans, implementation details, reflection.
- A project website: with links to all deliverables.
- A GitHub project Repo.
- A showcase entry.
- A record of project planning.
- A demonstration video.
- Presentation to a judging panel.

2.5 Acronyms

Acronyms	Description
ERR	Enhanced Reporting Requirements
ROS	Revenue Online Services
RBAC	Role-Based Access Control
PPSN	Personal Public Service Number
JWT	JSON Web Token
REST	Representational State Transfer
API	Application Programming Interface
SPA	Single Page Application
POC	Proof of Concept
CRUD	Create, Read, Update, Delete
GDPR	General Data Protection Regulation
UTC	Coordinated Universal Time
TAIN	Tax Agent Identification Number
X.509	Public Key Infrastructure Certificate Standard
PKCS#12	Public Key Cryptography Standard #12

3. Research and Analysis

3.1 Research

Research activities focused on three primary areas: regulatory requirements, existing systems, and technology evaluation. Revenue documentation and technical schema specifications were reviewed to understand mandatory submission structures and conditional reporting rules (Revenue, 2024a; Revenue, 2024b). This provided clarity on the data elements required for compliant ERR payloads and the role of the ROS API in the submission process. Technical research evaluated frontend and backend frameworks, and database technologies suitable for a SaaS-style application.

3.2 Analysis

The analysis stage translated research findings into planning and system requirements. Functional requirements were identified for user authentication, RBAC, expense claim capture and management, ERR payload generation, ROS API submission generation, and submission status reporting. These requirements were expressed as user stories to support iterative development.

3.3 Existing Solutions (Internal and External)

Within SD Worx, the standard SaaS development stack consists of Angular for frontend application development, ASP.NET Core for backend API and service construction, and C# as the primary programming language. This enterprise level stack supports large scale, production ready payroll and HR systems. For the purposes of this POC project, it was agreed that the application would be developed using a technology stack with which I had greater familiarity e.g. Svelte for frontend, Node.js for backend and MongoDB. The primary objective of the project was to explore and model the ROS ERR integration process, rather than replicate the organisation's enterprise framework. This ensured that the learning focus remained on regulatory modelling and system architecture rather than on mastering a new technology stack.

Within the SD Worx ecosystem, ERR processing is implemented as an embedded component of the existing payroll platform (MegaPay), rather than as a standalone system. The ERR workflow is tightly coupled with payroll execution, whereby expense data is captured during the timesheet creation process and subsequently included in the statutory submission cycle, this integrated approach simplifies processing by combining expense payments with payroll payments. However, this design inherently restricts ERR processing to payroll professionals. In contrast, a standalone expense management solution allows a broader range of users to manage and submit expense data independently of payroll.

Externally, the Irish payroll market contains a number of established platforms that support ERR compliance. These solutions can broadly be categorised into three groups: direct API submission platforms, automated file generation systems, and integrated accounting ecosystems. Direct API platforms such as Parolla, SimplePay and BullethQ (Revenue, 2024b; Vendor documentation, 2025) submit ERR data directly to Revenue via REST API integration, abstracting authentication, certificate handling and submission validation from the end user. In contrast, systems such as Strandum, Capture Expense and ERR Bridge generate ROS-compliant XML files which must be manually uploaded through the ROS portal. Mainstream accounting platforms such as Xero and QuickBooks typically rely on specialised Irish payroll add-ons to manage the ERR submission process.

Based on these internal and external considerations, the solution adopted in this project was a standalone, web-based application designed specifically to model the ERR submission lifecycle. This approach enables separation from payroll processing and supports broader user interaction.

While not representative of SD Worx's production architecture, this design choice is appropriate for a POC, as it prioritises modelling the submission lifecycle within a layered RESTful web application. By concentrating on schema construction, employee identification rules, and submission reference generation, the project demonstrates the architectural and integration principles required to support compliant ERR reporting.

3.4 POC Functional Requirements

Key functional requirements implemented in the prototype were:

- User registration/login and session management.
- RBAC (e.g., processor, admin).
- Create/read/update (CRUD) for employee, elements, company details and expense claims.
- ERR submission generation from pending claim records.
- ROS API integration to submit ERR payloads and capture response status.
- Reporting and audit trail of submissions.

3.5 Technology Choices

Prior to selecting specific technologies, a set of non-functional requirements was established to guide the architectural decisions. The system required:

- Support for secure API integration with external services (ROS)
- Ability to handle asynchronous communication and signing workflows
- Rapid development capability suitable for a POC
- A flexible data model to accommodate evolving schemas
- A lightweight, responsive UI
- Clear separation of concerns between frontend, backend, and external services

Based on these requirements, the following technology stack was selected. Svelte was chosen for frontend development due to its lightweight reactive architecture and minimal runtime overhead, enabling rapid development of a responsive user interface (Svelte, n.d.). Node.js was selected for the backend as its non-blocking I/O model is well suited to handling asynchronous operations, including external API communication and interaction with the signing microservice (Node.js, n.d.).

MongoDB was chosen as the database solution due to its document-oriented structure, which supports flexible schema design. This was particularly beneficial for modelling ERR payload structures during the prototyping phase, where requirements evolved iteratively (MongoDB, n.d.).

Overall, the selected technologies align with the defined non-functional requirements by supporting integration-focused processing, rapid iteration, and modular system design.

3.6 ROS API Integration Considerations

The initial ROS Handshake was implemented first and later developments integrated the submission payload creation and response handling.

5. System Design

5.1 Architecture Model

The system followed a tiered architecture: Svelte SPA for the UI, Node.js REST API for business logic, and MongoDB database for persistence. The ROS API was accessed via a dedicated integration service within the Node.js backend.

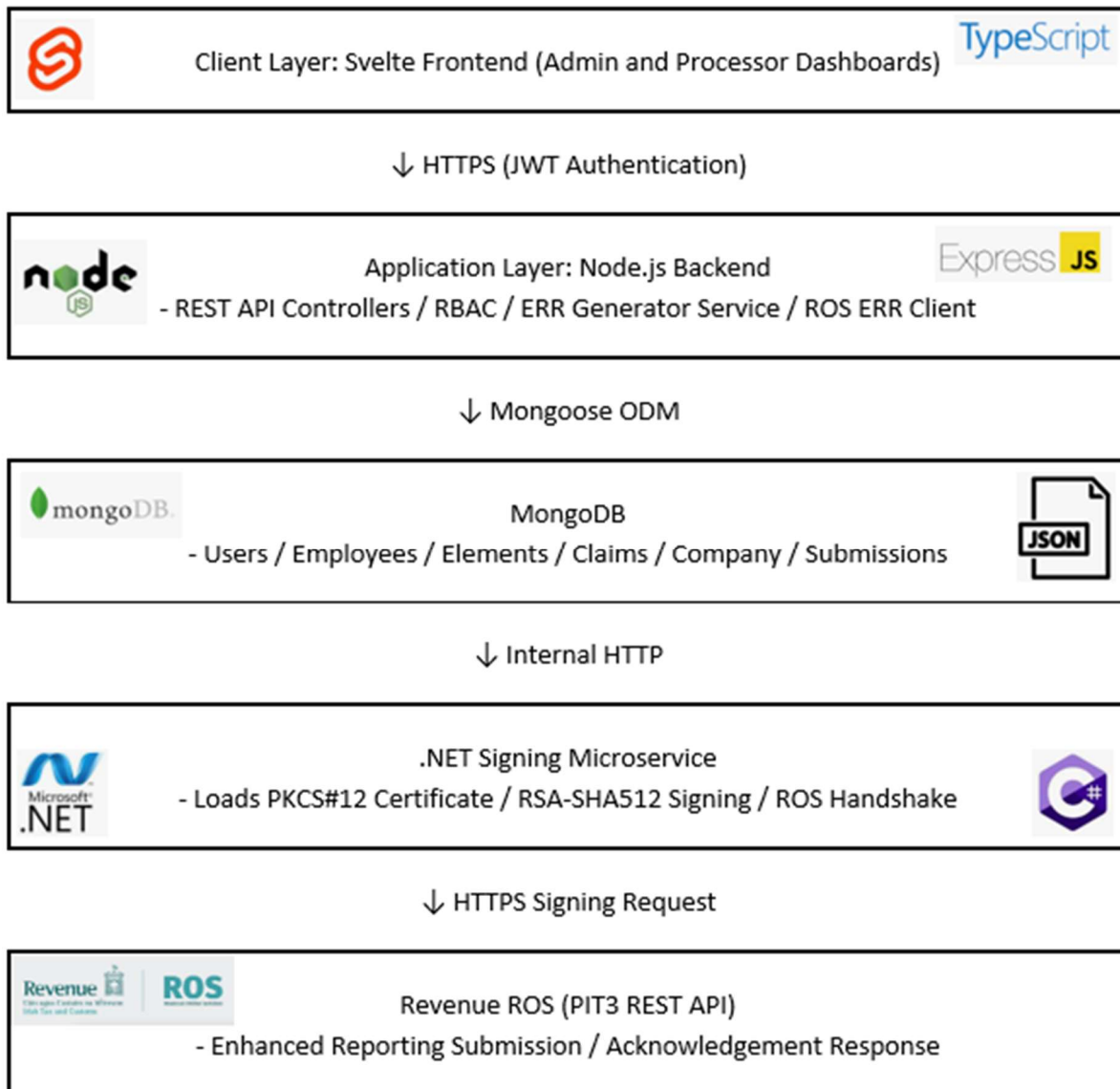


Figure 2: Architecture Model

5.2 Data Model

The data model was designed to support the generation of compliant submission payloads. A key challenge of the project was aligning data structures with Revenue defined schemas, which impose strict requirements on formatting.

To address this, a document-based data model using MongoDB was implemented for schema flexibility during prototyping, unlike a relational model, MongoDB allows incremental adaptation of document structures, making it suitable for a POC.

The data model was centred around the Claim entity. Claims capture expense data at the point of entry and are later transformed into submission payloads. A key design decision was the separation of audit fields (createdAt, updatedAt) from the statutory reporting field (payDate). This ensured the system could maintain an immutable audit history while generating submissions based on reporting dates required by Revenue.

Collections were designed to reflect distinct functional responsibilities:

- Employees store identity, address data, conditional identification logic whereby employees may be identified using a PPSN or alternative identifiers.
- Elements define expense categories and rates, aligning with Revenue classifications.
- Companies store employer-level data required for submissions.
- Users manage authentication and RBAC.

In addition, a dedicated Submission collection was introduced to persist the outcome of submission attempts. This includes generated payload data, acknowledgement responses, and trace identifiers.

Overall, the data model was designed to balance regulatory compliance and audit requirements, ensuring that internal data structures could accurately map to external ERR submission schemas.

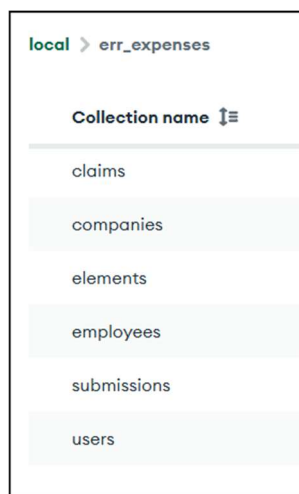


Figure 3: Database Collection

Collections and Key Fields (MongoDB Document Model):

- **claims**
_id (ObjectId), processor (ObjectId), amount (Number), description (String), payDate (Date), status (String), employee (ObjectId), element (ObjectId), days (Number), createdAt (Date), updatedAt (Date)
- **companies**
_id (ObjectId), employerRegistrationNumber (String), payrollReference (String), taxYear (Number), softwareUsed (String), softwareVersion (String), rosCertPath (String), rosCertPassword (String), agentTain (String), active (Boolean), createdAt (Date), updatedAt (Date)
- **elements**
_id (ObjectId), category (String), subCategory (String), description (String), active (Boolean), rate (Number), createdAt (Date), updatedAt (Date)
- **employees**
_id (ObjectId), firstName (String), familyName (String), employmentID (String), employeePpsn (String), employerReference (String), dateOfBirth (Date), address (Embedded Object), active (Boolean), createdAt (Date), updatedAt (Date)
- **submissions**
_id (ObjectId), employerRegistrationNumber (String), taxYear (Number), enhancedReportingRunReference (String), submissionID (String), payDate (Date), claims (ObjectId[]), submittedBy (ObjectId), lineItemCount (Number), totalAmount (Number), expensesBenefits (Array<Object>), acknowledgementStatus (String), acknowledgementID (String), revenueResponse (Object), traceId (String), createdAt (Date), updatedAt (Date)
- **users**
_id (ObjectId), name (String), email (String), password (String), role (String), createdAt (Date), updatedAt (Date)

5.3 UI Designs and Workflow

The system follows a structured user workflow that reflects the lifecycle of ERR processing, from data entry through to submission and acknowledgement. The primary user journeys are role-based and can be summarised as follows:

Processor Workflow:

1. User logs via the login screen
2. User creates expense claims by pay date, employee, element and amount
3. Claims are stored with a *pending* status
4. System generates an ERR compliant payload
5. Acknowledgement response is received, stored and claim statuses are updated to submitted
6. User can manage employees, elements and company setup
7. User can monitor submission history

Admin Workflow:

1. User logs in with administrative privileges
2. User can operate as a Processor (see Processor Workflow)
3. User can manage user accounts
4. User can monitor claim and submission activities
5. User can test Revenue integration (handshake, signing service)

Login



RBAC Check



Processor

Admin

Admin



Processing Functionality

Processing Functionality

Administrative Functionality



Create Claim (Store as Pending)

Create Claim (Store as Pending)

Manage Users



Manage Employees / Elements / Company details

Manage Employees / Elements / Company details

Revenue Handshake



Generate ERR Submission (Send to ROS API)

Generate ERR Submission (Send to ROS API)

Admin Reporting



Receive Acknowledgement

Receive Acknowledgement



Update Claims + Store Submission Record

Update Claims + Store Submission Record



Processing Reporting

Processing Reporting

Figure 4: System User Flow

5.3.1 Home Screen (/home)

Being the initial landing page upon app launch, giving basic information about the app, containing log in button and “about” hyperlink (redirecting to the SD Worx Ireland website).

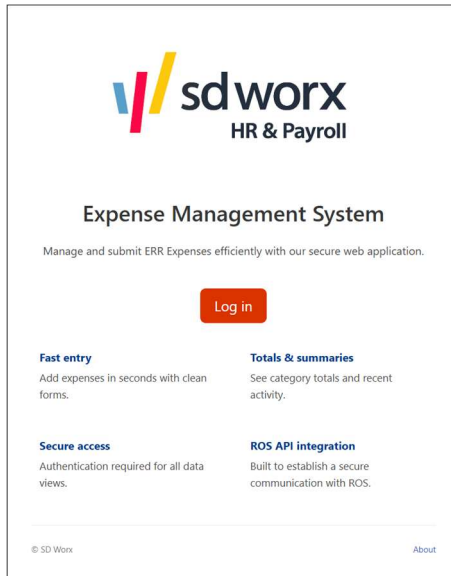


Figure 5: UI Landing Page

5.3.2 About hyperlink (/about)

The about hyperlink redirects to the main SD Worx Ireland website:



Figure 6: About Hyperlink

Redirect URL:

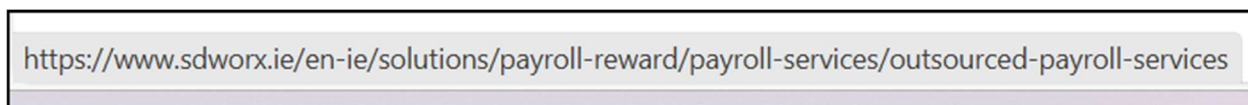


Figure 7: About URL

5.3.3 Log in Screen (/login)

The log in page where user credentials are entered, which also contains a hyperlink back to the home screen. Seeded log in credentials for two user roles:

Processor; processor@test.com / Processor123!

Admin; admin@test.com / Admin123!

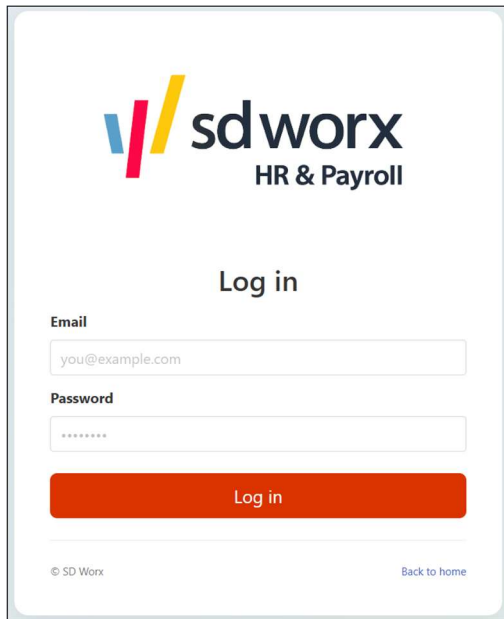
The image shows the login screen for SD Worx HR & Payroll. At the top is the SD Worx logo with the text 'HR & Payroll' below it. In the center, the text 'Log in' is displayed. Below this, there are two input fields: 'Email' with the placeholder 'you@example.com' and 'Password' with a masked password '*****'. A red 'Log in' button is positioned below the password field. At the bottom left, there is a copyright notice '© SD Worx', and at the bottom right, there is a 'Back to home' link.

Figure 8: Login Screen

Redirect button to return to /home



Figure 9: Back to Home Button

5.3.4 Initial log in for the Processor Role (/processor)

Upon successful authentication, users with the processor role are redirected to the main claims dashboard. This page serves as the central workspace for entering new claims, viewing or editing pending claims.



Figure 10: Expense Claims Menu Button (displaying logged in user)

Expense Claims

New Claim

Employee

Select Employee

Expense Type

Select Expense Type

Amount (€)

Paydate

dd/mm/yyyy

Description / Notes

Add Claim

Pending Claims for Submission

PAYDATE	EMPLOYEE	EXPENSE TYPE	DAYS	AMOUNT (€)
16/3/2026	Adele Dickinson	REMOTE_WORKING_DAILY_ALLOWANCE	2	6.40
16/3/2026	Adele Dickinson	TRAVEL_AND_SUBSISTENCE / EATING_ON_SITE	-	50.00

Figure 11: Processing Claims Dashboard

Edit pending claims; highlight and click on claim for editing:

Pending Claims for Submission

PAYDATE	EMPLOYEE	EXPENSE TYPE	DAYS	AMOUNT (€)
16/3/2026	Adele Dickinson	REMOTE_WORKING_DAILY_ALLOWANCE	2	6.40
16/3/2026	Adele Dickinson	TRAVEL_AND_SUBSISTENCE / EATING_ON_SITE	-	50.00

Figure 12: Select Claim for Editing

Edit pending claims; amend the necessary field/s and *Save Changes* or *Cancel* out of editing mode:

Pending Claims for Submission

PAYDATE	EMPLOYEE	EXPENSE TYPE	DAYS	AMOUNT (€)
16/3/2026	Adele Dickinson	REMOTE_WORKING_DAILY_ALLOWANCE	2	6.40

Employee

Adele Dickinson

Expense Type

REMOTE_WORKING_DAILY_ALLOWANCE

Pay Date

dd/mm/yyyy

Description

test

Days

2

Amount (€)

6.4

Save Changes **Cancel**

16/3/2026	Adele Dickinson	TRAVEL_AND_SUBSISTENCE / EATING_ON_SITE	-	50.00
-----------	-----------------	---	---	-------

Figure 13: Editing Pending Claim

5.3.5 Employee Management (/processor/employees)

This dashboard provides functionality for managing employee records. Users can create new employees or select an existing record for editing. The fields captured are the minimum mandatory employee identifiers for the submission record.

Employee Management

Add New Employee

First Name

Family Name

Employment ID

PPSN (Optional)

Employer Reference

Date of Birth

Address Line 1

City

County

Country

Add Employee

Employee List

NAME	EMPLOYMENT ID	EMPLOYER REF
Adele Dickinson	BEBDEB174E094C8BBB8C	EMP001
Sammy Rempel	AEBCEB164E194XBAJB9H	EMP002
Elaine Rice	CHAAIW561K212VPOCK8I	EMP003

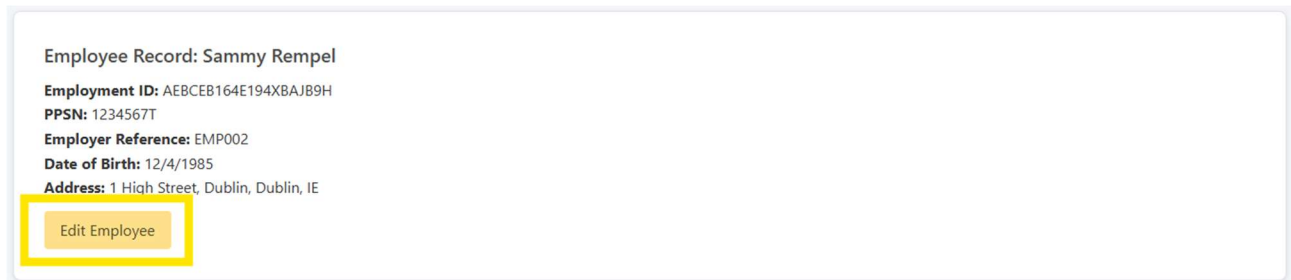
Figure 14: Employee Management Dashboard

View / Edit employee record; highlight and click on employee for viewing / editing:

Employee List		
NAME	EMPLOYMENT ID	EMPLOYER REF
Adele Dickinson	BEBDEB174E094C8BBB8C	EMP001
Sammy Rempel	AEBCEB164E194XBAJB9H	EMP002
Elaine Rice	CHAAIW561K212VPOCK8I	EMP003

Figure 15: Select Employee for Viewing / Editing

When the relevant employee record is selected, the record details are displayed, and the *Edit Employee* button becomes available:



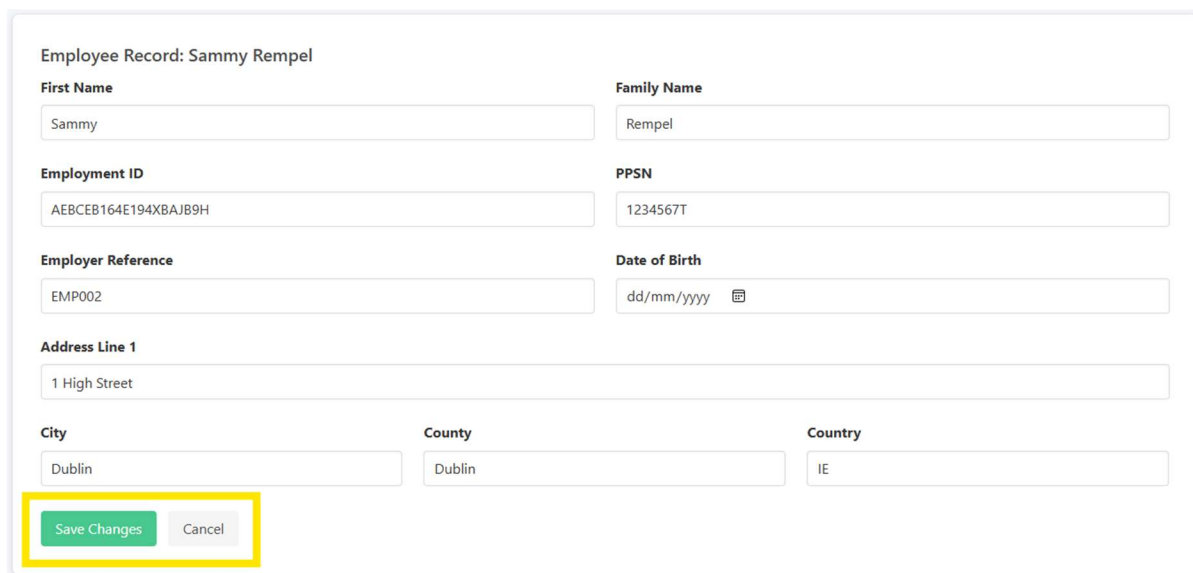
Employee Record: Sammy Rempel

Employment ID: AEBCEB164E194XBAJB9H
PPSN: 1234567T
Employer Reference: EMP002
Date of Birth: 12/4/1985
Address: 1 High Street, Dublin, Dublin, IE

[Edit Employee](#)

Figure 16: Edit Employee Option

Edit an employee record; amend the necessary field/s and *Save Changes* or *Cancel* out of editing mode:



Employee Record: Sammy Rempel

First Name	Sammy	Family Name	Rempel
Employment ID	AEBCEB164E194XBAJB9H	PPSN	1234567T
Employer Reference	EMP002	Date of Birth	dd/mm/yyyy
Address Line 1 1 High Street			
City	Dublin	County	Dublin
		Country	IE

[Save Changes](#) [Cancel](#)

Figure 17: Editing Employee Record

5.3.6 Element Management (/processor/elements)

This dashboard provides functionality for managing expense elements. Users can create new elements or edit existing records. Elements define the category, subcategory and days (where applicable) for compliant submission generation

Expense Elements

Add New Element

Category

Select Category

Description

Add Element

Active Elements

CATEGORY	SUBCATEGORY	DESCRIPTION	RATE
REMOTE_WORKING_DAILY_ALLOWANCE	-	Remote working daily allowance	3.2
SMALL_BENEFITS_EXEMPTION	-	Small benefits exemption	-
TRAVEL_AND_SUBSISTENCE	EATING_ON_SITE	Eating on site	-
TRAVEL_AND_SUBSISTENCE	EMERGENCY_TRAVEL	Emergency travel	-
TRAVEL_AND_SUBSISTENCE	SITE_BASED_EMPLOYEES	Site-based employees travel	-
TRAVEL_AND_SUBSISTENCE	SUBSISTENCE_UNVOUCHED	Subsistence unvouched	-
TRAVEL_AND_SUBSISTENCE	SUBSISTENCE_VOUCHED	Subsistence vouched	-
TRAVEL_AND_SUBSISTENCE	TRAVEL_UNVOUCHED	Travel unvouched	-
TRAVEL_AND_SUBSISTENCE	TRAVEL_VOUCHED	Travel vouched	-

Figure 18: Elements Management Dashboard

Edit element record; highlight and click on element for editing:

Active Elements			
CATEGORY	SUBCATEGORY	DESCRIPTION	RATE
REMOTE_WORKING_DAILY_ALLOWANCE	-	Remote working daily allowance	3.2
SMALL_BENEFITS_EXEMPTION	-	Small benefits exemption	-
TRAVEL_AND_SUBSISTENCE	EATING_ON_SITE	Eating on site	-
TRAVEL_AND_SUBSISTENCE	EMERGENCY_TRAVEL	Emergency travel	-
TRAVEL_AND_SUBSISTENCE	SITE_BASED_EMPLOYEES	Site-based employees travel	-
TRAVEL_AND_SUBSISTENCE	SUBSISTENCE_UNVOUCHED	Subsistence unvouched	-
TRAVEL_AND_SUBSISTENCE	SUBSISTENCE_VOUCHED	Subsistence vouched	-
TRAVEL_AND_SUBSISTENCE	TRAVEL_UNVOUCHED	Travel unvouched	-
TRAVEL_AND_SUBSISTENCE	TRAVEL_VOUCHED	Travel vouched	-

Figure 19: Select Element for Editing

The element record details are displayed and available for amending. Amend the necessary field/s and *Save Changes* or *Cancel* out of editing mode:

CATEGORY	SUBCATEGORY	DESCRIPTION	RATE
REMOTE_WORKING_DAILY_ALLOWANCE	-	Remote working daily allowance	3.2
SMALL_BENEFITS_EXEMPTION	-	Small benefits exemption	-
TRAVEL_AND_SUBSISTENCE	EATING_ON_SITE	Eating on site	-
TRAVEL_AND_SUBSISTENCE	EMERGENCY_TRAVEL	Emergency travel	-

Category

TRAVEL_AND_SUBSISTENCE

SubCategory

EMERGENCY_TRAVEL

Description

Emergency travel

Save Changes Cancel

Figure 20: Edit Element Option

5.3.7 Company Management (/processor/company)

This dashboard provides functionality for saving / editing company configuration data. It captures employer identifiers and parameters used in constructing submission payloads. For the purposes of this POC, certificate path and password fields are exposed in the UI; however, these would be securely managed in a production environment

Company Configuration

Employer Registration Number

8031508KH

Payroll Reference

ERI4

Tax Year

2026

Agent TAIN

88502T

ROS Certificate Path

C:\Users\elain\OneDrive\Documents\SETU HDip in Computer Science\Final Project\ERR-Expenses-Web-App-with-ROS-API-Integration\backend\certs\999966377.p12

ROS Certificate Password

Update Company

Figure 21: Company Details Dashboard

5.3.8 Submissions (/processor/submissions)

This dashboard provides a view of pending claims grouped by pay date. Users can select a reporting pay date to review the associated claims prior to submission.

Enhanced Reporting Submissions

Submit Pending Claims

Select Pay Date

Select Pay Date

Send Submission

Figure 22: Submission Dashboard

Select the relevant expense payment date to display the pending submission records:

Enhanced Reporting Submissions

Submit Pending Claims

Select Pay Date

16/3/2026

Select Pay Date

16/3/2026

Claims for 16/3/2026

EMPLOYEE	EXPENSE TYPE	DAYS	AMOUNT (€)
Adele Dickinson	REMOTE_WORKING_DAILY_ALLOWANCE	2	6.40
Adele Dickinson	TRAVEL_AND_SUBSISTENCE / EATING_ON_SITE	-	50.00

Figure 23: Display Pending Submission Line Items

Submitting pending claims: if the outstanding claims are ready for submission, click *Send Submission* otherwise return to the Claims dashboard to amend the submission record:

Enhanced Reporting Submissions

Submit Pending Claims

Select Pay Date

16/3/2026

Send Submission

Claims for 16/3/2026

EMPLOYEE	EXPENSE TYPE	DAYS	AMOUNT (€)
Adele Dickinson	REMOTE_WORKING_DAILY_ALLOWANCE	2	6.40
Adele Dickinson	TRAVEL_AND_SUBSISTENCE / EATING_ON_SITE	-	50.00

Figure 24: Send Submission Button

Submission success / failure: after clicking *Send Submission*, ROS will return either a successful (Submitted) or unsuccessful (Failed) status response. “Submitted” submission records are returned with an acknowledgment ID and “Failed” submission records are returned with the relevant validation error message:

Enhanced Reporting Submissions

Submission successfully acknowledged by ROS.

Submit Pending Claims

No pending claims available for submission.

Claims for 16/3/2026

EMPLOYEE	EXPENSE TYPE	DAYS	AMOUNT (€)
----------	--------------	------	------------

ROS Response
Submission ID: ERI4-202603-61d09905-ER1
Status: Submitted
ACK Status: ACKNOWLEDGED
ACK ID: e1a19c4d-66e6-41c6-b2a8-be9b6b7259b8

Figure 25: Submission Acknowledgement: “Submitted”

Enhanced Reporting Submissions

Submission failed. See details below.

Submit Pending Claims

Select Pay Date

16/3/2026

Send Submission

Claims for 16/3/2026

EMPLOYEE	EXPENSE TYPE	DAYS	AMOUNT (€)
Adele Dickinson	REMOTE_WORKING_DAILY_ALLOWANCE	2	6.40
Adele Dickinson	TRAVEL_AND_SUBSISTENCE / EATING_ON_SITE	-	50.00
Sammy Rempel	SMALL_BENEFITS_EXEMPTION	-	500.00

ROS Response
Submission ID: ERI4-202603-9366a089-ER1
Status: Failed

```
{
  "validationErrors": [
    {
      "description": "The value 'null' for property 'expensesBenefits[2].employeeID.employeeID' is not valid because must not be null.",
      "code": "N/A",
      "path": "expensesBenefits[2].employeeID.employeeID"
    },
    {
      "description": "The value 'null' for property 'expensesBenefits[2].employeeID.employeePpsn' is not valid because must not be null.",
      "code": "N/A",
      "path": "expensesBenefits[2].employeeID.employeePpsn"
    }
  ]
}
```

Figure 26: Submission Acknowledgement: “Failed”

In the current POC implementation, ERR submissions are processed as a single transactional unit. If any line item within a submission fails, the entire submission is rejected. As a result, all associated claims remain in a “pending” state. This simplifies error handling for the prototype. Future iterations would introduce line-item level validation and allow submission of valid records while isolating failures.

5.3.9 Processor Reports (/processor/reports)

This dashboard provides reporting functionality for submissions based on a selected pay date range (reset filter available). Metrics include total submissions, acknowledged / rejected submissions, and total monetary value submitted. A detailed submission history table is displayed. Future iterations would display the detailed line-items of each submission.

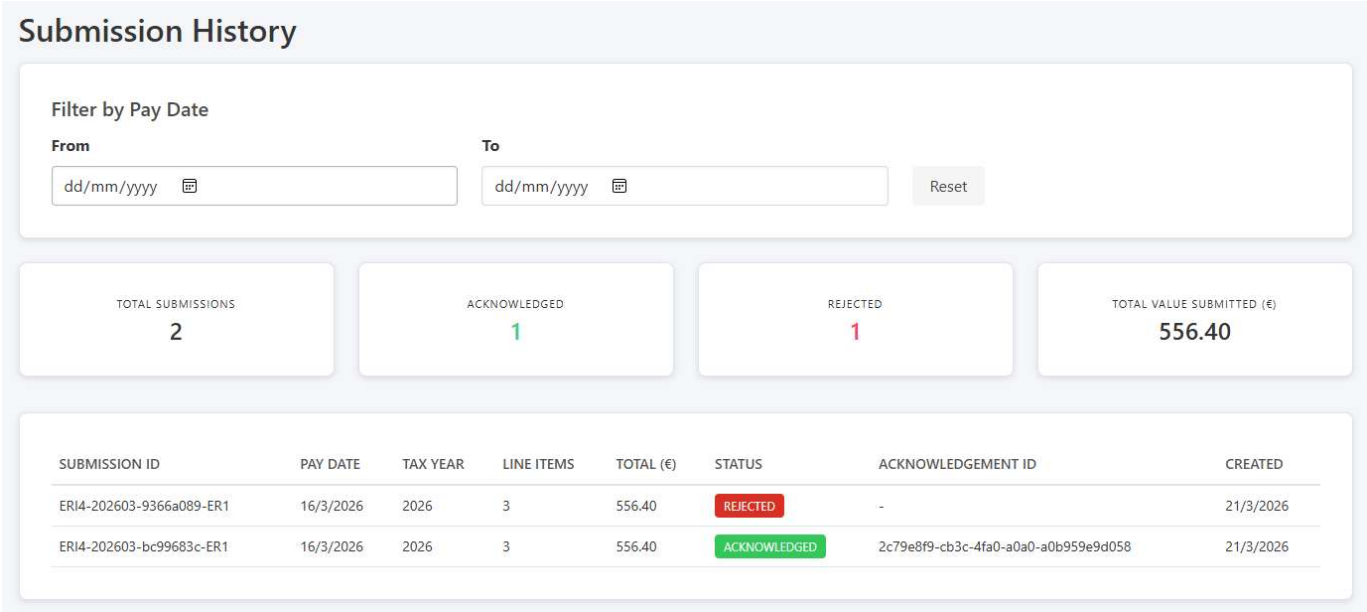


Figure 27: Processing Reporting Dashboard

5.3.10 Initial log in for the Admin Role (/admin)

Upon successful authentication, users with the admin role are redirected to the user management dashboard. This page includes creating, viewing, and updating user accounts (for future development). It provides oversight of roles and access levels within the application.

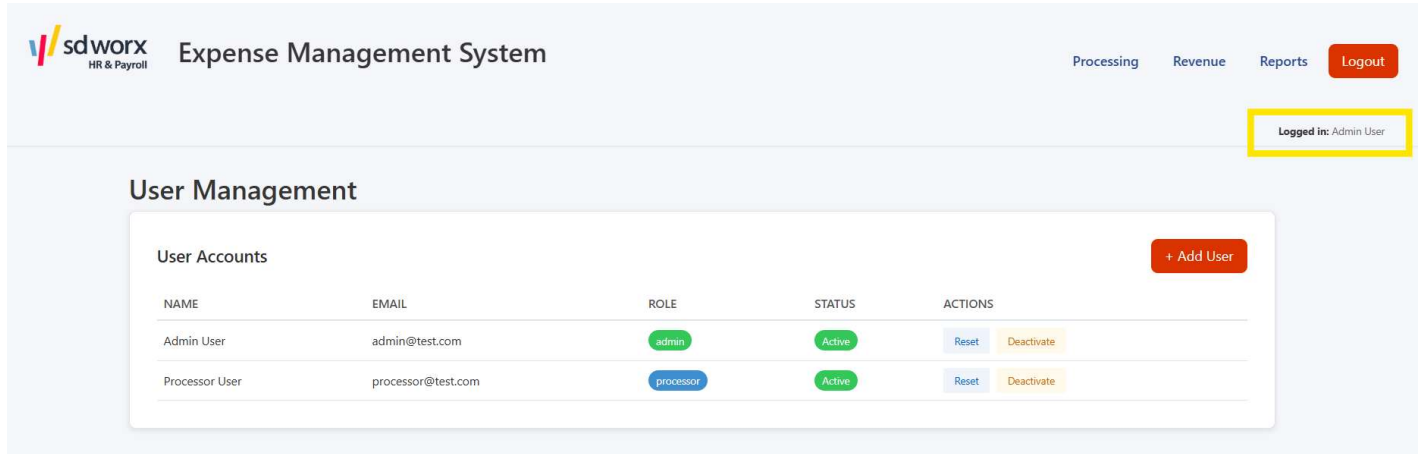


Figure 28: User Management Dashboard

5.3.11 Redirect to Processing Functionality (/processor (admin role))

This menu option allows an admin user to access the processor claims dashboard, providing full processing functionality.



Figure 29: Admin Redirect to Processing Functionality

From there, the admin user can navigate back to admin functionality:

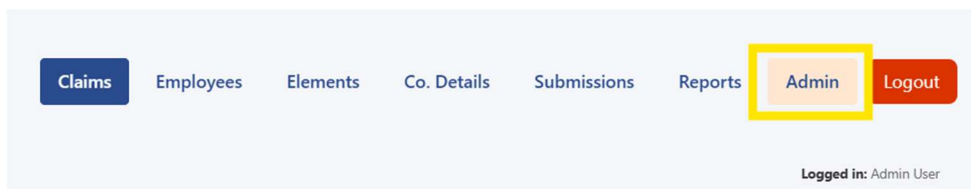


Figure 30: Dynamic Admin Menu Button

5.3.12 Revenue Integration (/admin/revenue)

This screen provides a dedicated interface for testing and validating the system's integration with ROS, specifically focusing on the cryptographic signing and handshake processes required for submissions. This screen exposes a set of controlled test actions that interact with both the Node.js backend and the .NET signing microservice. The Handshake function triggers the full backend-driven Revenue handshake process (/api/revenue/handshake), validating end-to-end communication with ROS.

The Health Check, Certificate Info, and Sign Test Data functions interact directly with the signing microservice. These endpoints isolate and verify critical components of the integration:

- Health Check (/health) confirms service availability.
- Certificate Info (/cert-info) exposes certificate metadata and Base64 encoding, supporting validation of certificate loading and configuration.
- Sign Test Data (/sign) validates the RSA-SHA512 signing process independently of the full ROS workflow.

Overall, this component acts as a diagnostic layer within the architecture, enabling verification of individual integration steps and reducing the risk associated with Revenue submission failures.

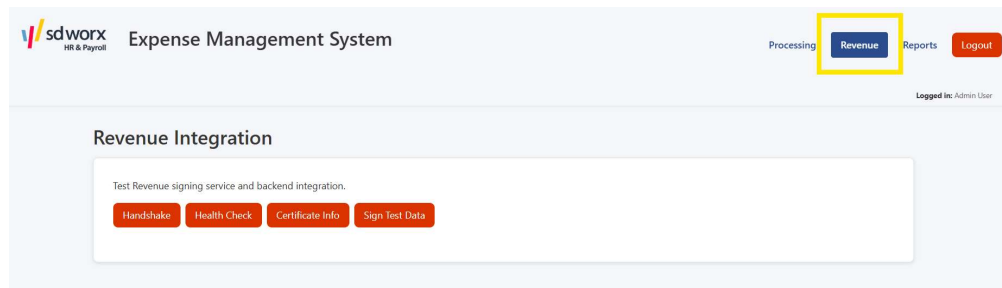


Figure 31: Revenue Integration Dashboard

Output screens for ROS Handshake attempts:

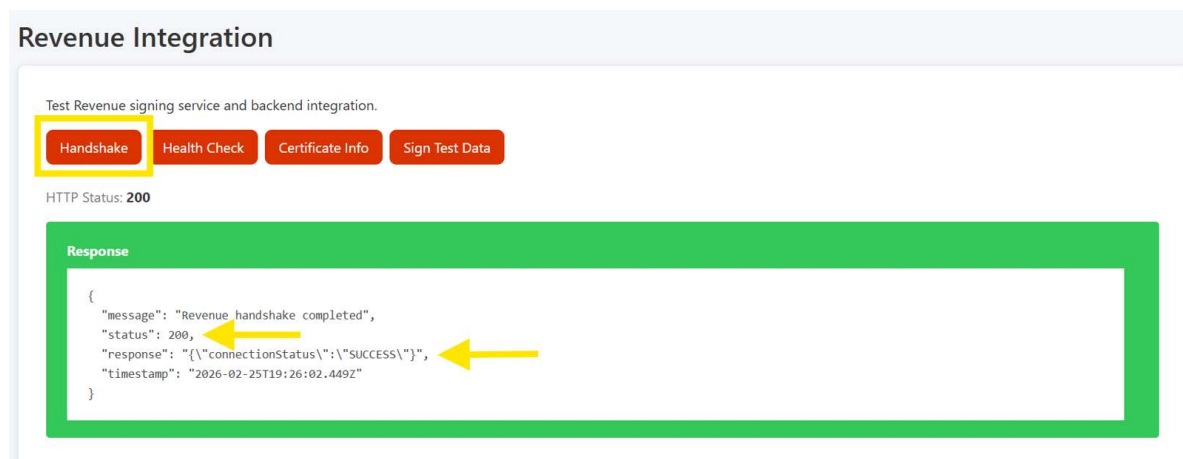


Figure 32: Successful ROS Handshake

Entered 18-Mar-26 15:23 by Revenue Agent

Good afternoon Elaine,

There was some configuration work carried out in PIT3 on the 14th of March - the permissions profiles of some certificates in PIT were updated. This explains the behaviour you are seeing which is unexpected. Your handshake requests should be working. We are speaking with the development team on this right now and will update you shortly when we have an exact timeframe of when the issue will be resolved. Apologies for this inconvenience.

Kind regards,
The PIT support team.

18-Mar-26 15:23 - E-mail sent to: Elaine Rice [elaine.rice@sdworx.com] from revenuehelpdesk@canfigure.cloud

Figure 33: Cert Issues; Revenues Confirmation

Revenue Integration

Test Revenue signing service and backend integration.

Handshake

Health Check

Certificate Info

Sign Test Data

HTTP Status: 200

Response

```
{
  "message": "Revenue handshake completed",
  "status": 403,
  "response": "{\\\"validationErrors\\\": [{\\\"description\\\": \\\"Cert does not have appropriate permissions.\\\", \\\"code\\\": \\\"1011\\\", \\\"path\\\": \\\"N/A\\\"}]}\",
  "timestamp": "2026-03-22T13:21:38.623Z"
}
```

Figure 34: Unsuccessful ROS Handshake

Output screen for Health Check:

Revenue Integration

Test Revenue signing service and backend integration.

Handshake

Health Check

Certificate Info

Sign Test Data

HTTP Status: 200

Response

```
{
  "service": "Revenue Signing Service",
  "status": "Running",
  "timestamp": "2026-03-22T22:07:11.2454477Z"
}
```

Figure 35: Signing Service Health Check Output

Revenue Integration

Test Revenue signing service and backend integration.

Handshake Health Check Certificate Info Sign Test Data

HTTP Status: **200**

Response

```
{
  "subject": "CN=2026, OU=999966377, O=2026, C=IE",
  "issuer": "CN=R05 CA 2030 - FOR TEST PURPOSE ONLY, OU=Revenue On-Line Service, O=Revenue Commissioners, C=IE",
  "notBefore": "2025-12-18T08:44:19+00:00",
  "notAfter": "2027-12-18T08:44:19+00:00",
  "thumbprint": "4D3AC640368ED219B5CF242AB9F83095D2AB0F5E",
  "base64": "MIIFejCCA2XgAwIBAgIUMB5C/EB1HNZTSeIyKu4nbyXwsXYwDQYJKoZIhvcNAQELBQAwFTELMAkGA1UEBhMCsUUXHjAcBgNVBAoMFVJldmVudWJgQ29tbWlzc2lvcmlvYyEgMB4GA1UECw"
}
```

Output screen for ROS Certificate RSA-SHA512 Signing:

Revenue Integration

Test Revenue signing service and backend integration.

Handshake Health Check Certificate Info Sign Test Data

HTTP Status: **200**

Response

```
{
  "signature": "T7M8PSe5FaobmXPxv82zNzgdzwQ35woj9iRrZa91CcnIA26t3SUEefhpugFgcVsQ19Bvx8MeZ680JW/1en4lpXijaAfqRi8o5VWmieBZabfxnj+DLJlQ0UtwcgVadB5453nGr8AEw..."
}
```

28

5.3.13 Admin Reports (/admin/reports)

This dashboard provides administrative oversight of both claim and submission activity across all users in the system (with filter resets). It supports filtering by user, status, and date ranges (both audit and statutory), enabling detailed analysis of system usage and processing.

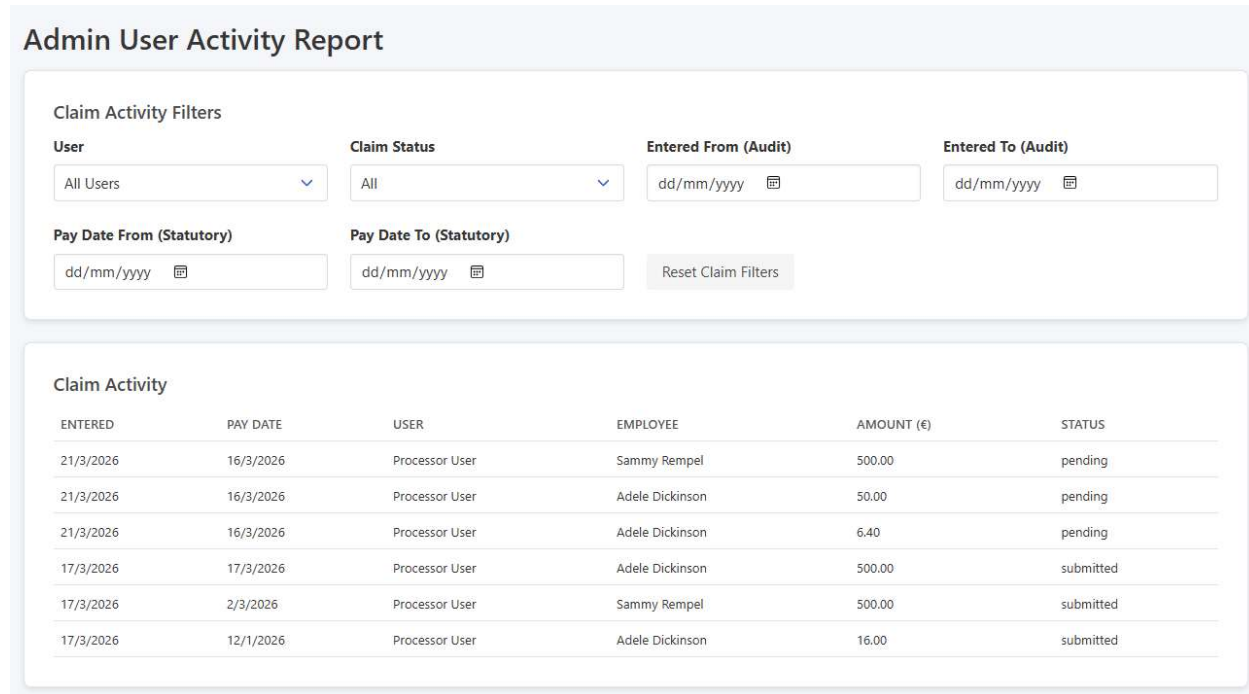


Figure 38: Admin Reports Dashboard; Claims Activity

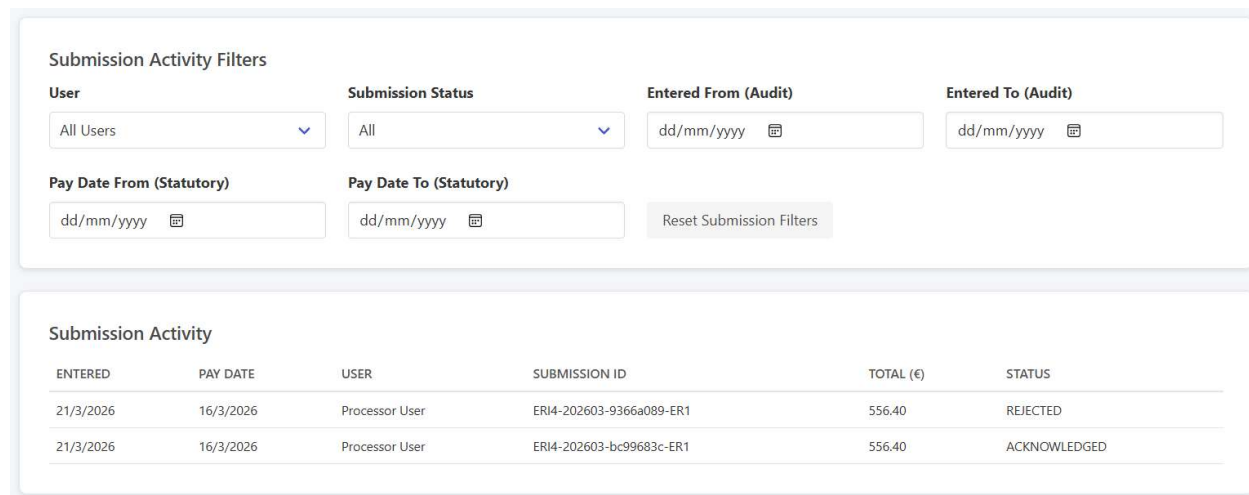


Figure 39: Admin Reports Dashboard; Submission Activity

5.3.14 Logout Menu Option (logout() → /login)

The logout option terminates the user session by clearing stored authentication credentials and redirects the user to the login page. This ensures that protected routes are no longer accessible without re-authentication. It supports basic session security and enforces RBAC within the application.



Figure 40: Logout Button

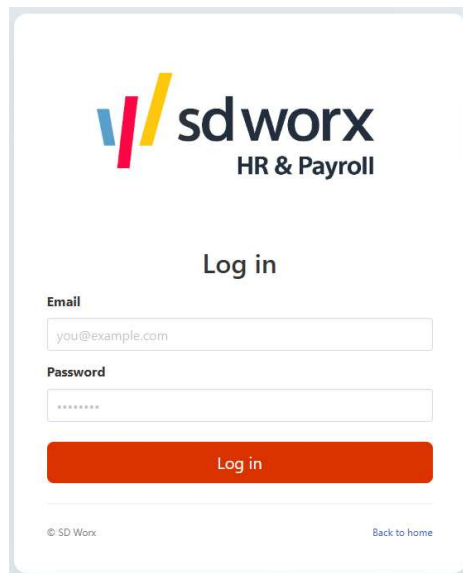


Figure 41: Logout Redirect to Login Screen

5.4 Security and Privacy by Design

Although the prototype uses non-sensitive test data, the system design incorporates controls aligned with production security practices. Authentication is implemented using JWT-based token handling with hashed passwords (bcrypt), and all protected routes enforce RBAC at middleware level. Input validation is applied at API level to reduce the risk of malformed data, consistent with OWASP secure coding principles (OWASP, 2021).

The architecture separates concerns by isolating cryptographic operations within a dedicated .NET signing microservice, ensuring secure handling of private keys and preventing exposure within the main application layer. Communication between components assumes HTTPS in a production environment, with secure certificate-based authentication required for ROS integration.

6. Implementation

6.1 Backend Architecture and Security (see Appendix A diagram)

The backend was implemented using Node.js and Express, structured modularly into controllers, middleware, services, and data models to ensure separation of concerns. Environment configuration is managed via dotenv, securing database credentials and JWT secrets. A dedicated database module establishes and verifies MongoDB connectivity at startup. Authentication uses JWT to support stateless sessions (Node.js, n.d.). On login, the server issues a signed token containing the user ID and role, which must be included in all protected requests. RBAC is enforced through middleware that validates JWTs and restricts role-specific endpoints.

Key REST endpoints include:

- /api/auth → authentication
- /api/admin → admin control
- /api/processor/claim → claim management
- /api/processor/submissions → ERR submission
- /api/employees → employee management
- /api/elements → element management
- /api/company → company configuration
- /api/revenue → revenue integration
- /api/reports → reporting
- /api/reports/submissions → submission reporting
- /api/admin/reports → admin reporting
- /api/users → user management
- /api/revenue/handshake → ROS handshake
- /api/submissions/generate → submission generation

6.2 Frontend Architecture and User Interface

The frontend was developed using SvelteKit, enabling a structured SPA with clear separation between routes, layouts, and reusable components. Routing is defined within the /routes directory and includes:

- /login (public)
- /admin (protected)
- /processor (protected)

Users are redirected dynamically after authentication based on role. A centralised design system inspired by SD Worx branding was implemented using CSS variables in app.css. Reusable .sdw-* classes standardise layout, forms, notifications, and tables. This approach avoids hard-coded styling and improves maintainability.

6.3 Claims Processing and Business Logic

Claims were designed to meet statutory ERR requirements. Each claim stores:

- createdAt timestamp (audit trail)
- payDate (statutory reporting date)

This separation ensures audit integrity while supporting Revenue's requirement that submissions be made on or before the pay date. Claims reference:

- Submitting user (audit traceability)
- Employee
- Payment categorisation

Validation ensures required fields (including pay date) are present before persistence.

6.4 ROS API Integration Layer

A dedicated backend service manages communication with the ROS API. The integration layer:

- Constructs ERR-compliant payloads
- Generates deterministic identifiers (run reference, submission ID, line IDs)

Employees with and without PPSNs are handled according to Revenue specifications (Revenue, 2024b; Revenue, 2024c).

6.5 ERR Submission Schema Design

A core objective was designing an ERR submission schema compliant with ROS specifications. Identifiers such as run reference, submission ID, and line item ID are generated deterministically from employer configuration and sequencing logic (Revenue, 2024c). Conditional employee identification rules were modelled explicitly:

- With PPSN: employment ID + PPSN
- Without PPSN: employer reference + address + date of birth

Structured seed datasets were introduced for employees, elements, company configuration, and submission metadata. This schema-first approach reduced integration risk and allowed submission logic to be validated independently of the UI.

6.6 Submission Modelling Integration

Claims were refined to support statutory reporting:

- Audit timestamp retained
- Pay date used for submission grouping

Claims were normalised to reference employees and elements. This improves data integrity and supports transformation into payloads.

Submission output is:

- Returned via API
- Persisted to disk as JSON

UI enhancements were intentionally deferred until submission logic was validated.

6.7 ERR Submission Generation Engine

Development focus shifted to the submission engine due to its regulatory and technical significance. A dedicated service assembles ERR payloads by combining claim, employee, element, and company data. Submissions are scoped to a single pay date. Conditional reporting logic enforces:

- PPSN-based vs non-PPSN employee rules
- Category/sub-category specific requirements

6.8 Data Normalisation and End-to-End Validation

Earlier claims created, before the schema refinement, required realignment with updated employee and element references. Date handling was normalised during submission generation to account for MongoDB UTC timestamps while producing Revenue compliant date only values (per ISO 8601 standards – ISO, 2019). End-to-end submission testing confirmed deterministic identifier generation and valid ERR payload structure.

6.9 Conditional Revenue Field Handling

Revenue requires category specific conditional fields. For example:

- REMOTE_WORKING_DAILY_ALLOWANCE requires numberOfDays and amount
- Other categories require amount only

Submission logic was extended to enforce conditional fields. Invalid conditional data prevents submission generation. Claims were enhanced to support optional day counts without affecting other expense types. The system now correctly generates mixed category submissions within a single pay date.

6.10 Design of the .NET Signing Microservice

The purpose of the .NET Signing Microservice is to handle Revenue ROS cryptographic operations that cannot be reliably implemented in Node.js. Specifically, the microservice is responsible for:

- Loading the Revenue-issued ROS X.509 certificate (PKCS#12)
- Applying Revenue specific password handling
- Performing RSA-SHA512 signing operations
- Returning cryptographic artefacts to the Node.js backend

All business logic, data modelling, and submission generation remain within the Node.js application. The .NET service is intentionally stateless, minimal, and isolated. This separation mirrors enterprise payroll systems where cryptographic operations are delegated to trusted components.

6.11 Revenue ROS Certificate Cryptographic Signing Engine Implemented

Initial attempts to handle the ROS PKCS#12 certificate directly within Node.js exposed compatibility and cryptographic parsing limitations, a minimal ASP.NET Web API microservice was designed and implemented to act as a dedicated cryptographic signing service. The ROS PIT3 certificate (01536466V.p12) was successfully loaded using X509Certificate2 with Revenue's required Base64(MD5(password)) transformation.

A secure /sign endpoint was implemented that performs RSA-SHA512 signing using the certificate's private key and returns a Base64 signature compliant with the Revenue HTTP Signature specification. Testing via Postman confirmed correct generation of RSA-SHA512 signatures, demonstrating a fully functional signing engine aligned with the PIT3 API documentation.

6.12 Revenue PIT3 handshake fully operational using .NET signing microservice

Initial handshake attempts with the Revenue ROS PIT3 environment returned validation error code 1011 ("Cert does not have appropriate permissions"), indicating that the certificate was valid cryptographically but not authorised for web service access. Consultation with ROS Developer Support confirmed that the issue was related to certificate registration and associated employer and agent identifiers rather than the signing implementation.

A new Revenue PIT3 certificate (999966377.p12) was issued, associated with Employer Registration Number 8031508KH and Agent TAIN 88502T. The certificate credentials were updated within the system configuration. The handshake request parameters were amended to include the correct employer registration number and agent TAIN, ensuring alignment between certificate ownership and submission metadata.

Following these updates, the handshake process returned a successful authenticated response with connectionStatus = SUCCESS. This confirms that the certificate permissions, RSA-SHA512 signature generation, canonical request construction, HTTP signature header formation, and ROS PIT3 endpoint authentication are all functioning correctly. The .NET signing microservice successfully performs Revenue-compliant cryptographic operations, and the Node.js backend correctly generates and signs authenticated requests.

6.13 ERR Submission Lifecycle and ROS API Integration

Following successful implementation of the PIT3 handshake, the next stage involved designing and validating the full ERR submission lifecycle.

A real world lifecycle would comprises multiple status transitions, reflecting both internal processing and external acknowledgement by Revenue:

Status	Meaning	Trigger
pending	Claim entered in system	Processor action
generated	ERR JSON payload file created for manual upload to ROS	automatic (export function)
acknowledged	Revenue confirmed receipt (re JSON file)	ROS response (ROS System)
submitted	Successfully transmitted to ROS API	API call
acknowledged	Revenue confirmed receipt (API)	ROS response (API)

For the purposes of this POC, a simplified workflow was implemented:

When a user selects “Submit to ROS”, the system performs the following sequence (initially simulated on postman):

1. Generates JSON file with submission payload
2. Persists JSON file to disk (manual upload fallback)
3. Attempt ROS API submission
4. If API succeeds, update claims status to “submitted”
5. If API fails, claims remain pending (JSON file is available for manual upload)

This ensures that a JSON file is always produced, the API submission is attempted, and the claim status is only updated upon success.

This separation of responsibilities was implemented using two distinct backend services:

Service 1 – Generate Submission Service

1. Builds ERR compliant JSON payload
2. Validates schema structure
3. Exports file to /exports directory
4. Returns payload to caller
5. Does not modify database state

Service 2 – Submit to ROS Service

1. Constructs HTTP request in accordance with Revenue PIT3 REST requirements
2. Applies SHA-512 hashing and Base64 encoding for the Digest header
3. Applies RSA-SHA512 signature via .NET signing microservice
4. Submits request to ROS endpoint
5. On HTTP 200 response → updates claim status to “submitted”

Build and Test Stages

The implementation progressed in four structured stages:

1. ROS configuration and environment variables (Server starts)
 - a. Certificate registration
 - b. Environment variables configured
 - c. Server startup validation
2. ROS client service (Node → .NET signing microservice) (Manually test endpoint)
 - a. Canonical string construction
 - b. Signature generation validation
 - c. Header formation testing
3. ROS submission service (Node → ROS API) (Mock submission)
 - a. Mock submissions
 - b. Simulated success responses
 - c. Controlled database update testing
4. Integrate ROS submission into ERR workflow (Full submission)
 - a. End-to-end submission
 - b. Status transitions validated
 - c. Error handling scenarios tested

Digest Mismatch Error Investigation

During live API testing, the following error was returned:

“Digest header must match digest that has been regenerated from request body content.”

Revenue recalculates the SHA-512 hash of the received byte sequence and compares it to the transmitted Digest header (as specified in the PIT3 REST API documentation), this mismatch indicated that the transmitted body and hashed content were not identical at byte level.

Initial triage included (OpenAI, 2026):

1. Verification of payload length
 - a. ROS request payload: 791 bytes
 - b. Exported JSON file: 1411 bytes

Confirmed they were different representations and not directly related

2. Digest debugging
3. Canonical signature reconstruction analysis
4. Header validation
5. Date header correction (RFC-7231 format required rather than ISO-8601)
6. Byte-level comparison of signed payload

After three days of analysis and corrections, the remaining issue was configuration related rather than cryptographic. The employer registration number embedded in the submission metadata did not match the employer registration number associated with the newly issued PIT3 certificate resulting in the byte difference and caused ROS to reject the request.

After updating the database Company configuration document to:

- Employer Registration Number: 8031508KH
- Agent TAIN: 88502T
- Certificate: 999966377.p12

The submission request returned a successful authenticated response:

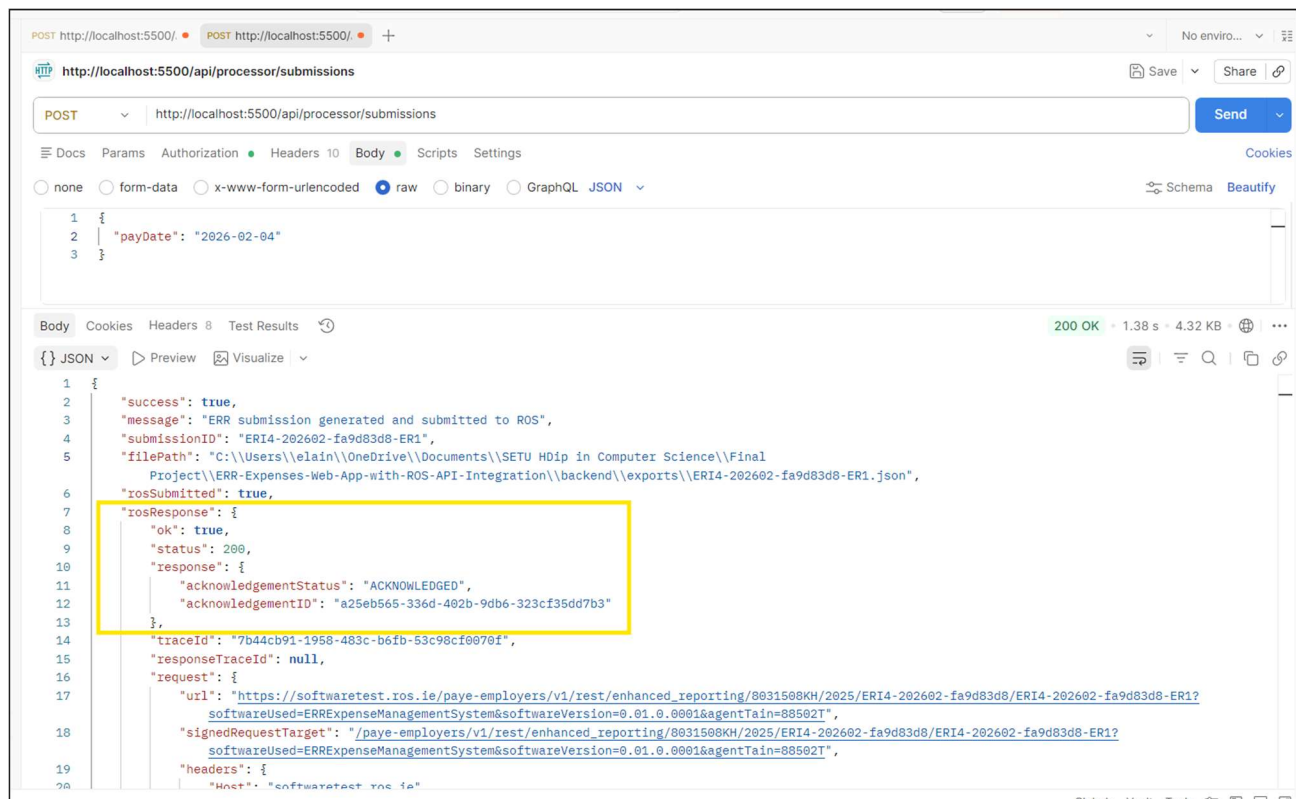


Figure 42: Successful ROS Submission Acknowledgement

This phase confirmed that:

- ERR JSON generation conforms to Revenue's schema
- SHA-512 digest generation aligns with HTTP Digest header requirements
- RSA-SHA512 signature creation is compliant with ROS HTTP Signature specification
- Canonical request construction is valid
- The .NET signing microservice and Node.js backend operates correctly as an integrated system

6.14 ERR Submission Lifecycle from UI

While the ERR submission workflow was completed and tested from postman, the next focus was on ensuring end-to-end submission capability and data persistence from within the system.

ERR Submission Generation

The backend service responsible for generating ERR submissions was refined to:

- Dynamically generate unique enhancedReportingRunReference and submissionID values
- Aggregate pending claims by pay date
- Calculate submission summary metrics including:
 - o LineItemCount
 - o totalAmount
- The system now ensures that duplicate submissions are prevented through unique identifier generation.

Database Persistence & Audit Trail

Implemented during this session was the persistence of submission records in MongoDB. A Submission model was finalised to store:

- Employer Registration Number
- Tax Year
- Enhanced Reporting Run Reference
- Submission ID
- Associated Claim IDs
- Line item count
- Total submission value
- Full snapshot of expensesBenefits payload
- Revenue acknowledgement details
- Revenue trace ID
- Complete ROS response object

The design ensures that each submission attempt is audit-safe and traceable, supporting compliance and future reporting requirements.

Claim Workflow State Management

The claim lifecycle logic was finalised:

- Claims with status "pending" are included in submission generation
- Claims are updated to "submitted" only upon successful ROS acknowledgement
- Failed submissions do not alter claim status

This guarantees data integrity between claim state transitions.

Frontend Integration

A frontend submission interface was implemented allowing:

- Selection of claims by pay date
- Display of ROS acknowledgement response
- Confirmation of submission status to the user

This completes the functional workflow from claim entry to ROS submission and response.

6.15 Employee Management from UI

The system was extended to support employee management rather than relying on pre-seeded database records.

Backend Enhancements

A new REST endpoint (POST /api/employees) was implemented to allow users create employee records. The controller performs validation of required fields:

- First name
- Family name
- Employment ID
- Employer reference
- Date of birth
- Structured address object

Validated employee records are persisted to MongoDB using the Employee model.

Frontend UI Improvements

The employee management interface was designed to align with existing forms, applying Bulma styling to ensure visual consistency.

6.16 Element Management from UI

The system was extended to support users creating element records, replacing the previously seeded configuration approach.

Backend Enhancements

A new REST endpoint (POST /api/elements) was implemented to allow authenticated users to create reporting elements directly from the UI. The controller performs validation of required fields:

- Category (supporting Revenue enum values)
- Description (UI facing label)
- Optional subcategory (applicable to TRAVEL_AND_SUBSISTENCE)
- Optional rate (applicable to REMOTE_WORKING_DAILY_ALLOWANCE)

New element records are persisted to MongoDB using the Element model. Elements are retrieved via GET /api/elements and sorted hierarchically by category and subcategory.

Frontend UI Improvements

A new processor facing interface (/processor/elements) was developed to enable:

- Creation of new reporting elements
- Display of elements in a structured table
- Conditional rendering of the daily rate field when the category is REMOTE_WORKING_DAILY_ALLOWANCE
- Conditional rendering of sub-categories field when the category is TRAVEL_AND_SUBSISTENCE

A controlled dropdown was implemented for sub-category selection to enforce compliance with Revenue-defined enum values. Bulma styling was applied to ensure visual consistency.

6.17 Company Configuration from UI

The system was extended to support dynamic management of company configuration data, replacing the previously database seeded configuration. This enhancement ensures that organisational metadata can be maintained directly through the user interface.

Backend Enhancements

New REST endpoints were implemented:

- GET /api/company
- POST /api/company
- PUT /api/company/:id

The controller manages a single active company configuration and performs validation of required fields:

- Employer Registration Number
- Payroll Reference
- Tax Year
- ROS Certificate Path
- ROS Certificate Password

Validated company configuration records are persisted to MongoDB using the Company model, stored in the companies collection.

This configuration is used at runtime for:

- ERR run reference generation
- Tax year assignment
- Software metadata inclusion
- Agent TAIN submission logic
- ROS certificate digital signature handling

Frontend UI Improvements

A new processor-facing interface (/processor/company) was developed to allow:

- Viewing existing company configuration
- Creating a new configuration (if none exists)
- Updating existing configuration

Sensitive fields such as the ROS certificate password were handled using a masked input field (type="password").

Bulma styling was applied to ensure visual consistency.

6.18 System Reporting and Governance Layer

Reporting functionality was implemented:

For processors, the /processor/reports interface focuses on submission history. An interface was developed to display all submissions stored in the submissions collection. Each record includes submission identifiers, pay date, tax year, line item count, total value submitted, acknowledgement status, acknowledgement ID and the date the submission was sent. Date range filtering was implemented based on pay date, allowing processors to review reporting activity for specific payroll periods.

For administrators, a more governance reporting layer was introduced. The /admin/reports interface provides user activity visibility across two distinct domains: claim activity and submission activity. Claims represent operational transactions entered by processors, whereas submissions represent statutory filings sent to Revenue. Importantly, rejected submissions are excluded from acknowledged value totals to ensure financial summaries reflect only successfully submitted statutory amounts.

Each section includes independent filtering controls:

- User based filtering (to identify activity by individual system users)
- Status filtering (claim workflow status or submission acknowledgement status)
- Audit date filtering (Entered From → Entered To, based on createdAt)
- Statutory date filtering (Pay Date From → Pay Date To)

The distinction between audit dates and statutory pay dates was a deliberate design decision. Audit dates enable administrators to review when records were created within the system. Statutory pay dates reflect the legal reporting period associated with submissions.

To support accountability, the Submission schema was extended to include a submittedBy reference to the User model enabling traceability of which user triggered each submission. Combined with the existing processor reference on claims, the system now provides a complete audit trail linking users to both individual transactions and statutory filings.

6.19 Edit Functionality for Employees, Elements and Claims (CRUD Completion)

The system was extended to support update capability for Employees, Elements, and Claims. This enhancement completes the Create / Read / Update lifecycle for operational records.

Backend Enhancements

New RESTful PUT endpoints were introduced to allow users update existing records. For Employees, the system uses findByIdAndUpdate.

For Elements, update functionality dynamically reapplies sub-category specific validations:

- Subcategory selection remains for TRAVEL_AND_SUBSISTENCE only.
- Rate values remain for REMOTE_WORKING_DAILY_ALLOWANCE only.

For Claims, the update functionality was intentionally restricted to claims with a status of pending only. Claims that have been generated or submitted cannot be modified. This safeguards against retrospective alteration of statutory data.

Frontend Enhancements

The user interface was enhanced to support inline editing within tables rather than redirecting to separate pages. Functionality:

- Select the relevant record and click “Edit Employee” to activate edit mode (for employees).
- Select the relevant record to activate edit mode (for elements and claims).
- Form fields are pre-populated with existing values.
- Updates are persisted via authenticated PUT requests.
- Lists refresh dynamically after successful updates.

Future development would address stronger validations around editing.

7. Testing and Validation

7.1 Test Strategy

Testing focused on validating the end-to-end submission workflow. The strategy combined API-level testing of REST endpoints, UI-level testing of user walkthroughs, and integration testing of the ROS submission process. Particular emphasis was placed on verifying the correctness of payload generation and HTTP signature construction. Security testing included validation of authentication (JWT), RBAC, and restricted endpoint access.

7.2 Sample Test Cases

ID	Scenario	Expected Result	Outcome
TC-01	User logs in with valid credentials	JWT issued; user redirected based on role	Pass
TC-02	Create new expense claim	Claim persisted and visible in pending list	Pass
TC-03	Edit existing claim	Updated values saved and reflected in UI	Pass
TC-04	Generate ERR submission	JSON payload generated with correct totals and structure	Pass
TC-05	Submit ERR to ROS (test endpoint)	Signed request accepted; acknowledgement returned	Pass
TC-06	ROS submission failure	Submission stored; claims remain in "pending" state	Pass
TC-07	Signing service health check	Service responds with status "Running"	Pass
TC-08	Certificate retrieval	Certificate metadata returned successfully	Pass
TC-09	Signature generation test	Valid RSA-SHA512 signature returned	Pass
TC-10	Unauthorized endpoint access	Access denied; redirect or error response	Pass

8. Results and Evaluation

The prototype successfully demonstrates an end-to-end workflow from expense capture, payload generation, and submission processing. Core user journeys were validated across both processor and administrator roles.

From an architectural perspective, the system achieves a clean separation of concerns through a layered design, with a stateless REST API, a dedicated cryptographic signing microservice, and a decoupled frontend. This modular approach aligns with SaaS deployment models, isolating complex ROS integration logic from core business functionality.

A key outcome of the project is the successful implementation of the submission generation engine and production of Revenue compliant payloads, with signed requests successfully processed and acknowledgement responses captured and persisted.

Overall, the results demonstrate that the proposed architecture is capable of supporting compliant ERR reporting workflows and provides a good foundation for future enhancements.

9. Critical Self-Review, Reflection and Future Improvements

9.1 What Was Achieved

A functional POC was successfully developed to demonstrate end-to-end ERR submission workflow. The solution enables users create and manage expense claims and generate Revenue compliant payloads. Integration with ROS was implemented, with responses captured and persisted to provide status visibility and a complete audit trail of submission activity.

9.2 What Was Learned

The project provided practical experience across the full-stack development lifecycle, including the implementation of reactive user interfaces using Svelte, the design of RESTful APIs in Node.js, and document-based data modelling in MongoDB. It also involved the integration of an external API, requiring an understanding of HTTP signatures, cryptographic signing, and structured payload generation. In addition, the project reinforced key security and architectural principles.

9.3 Problems Encountered and Solutions

During the implementation, a technical challenge arose from the interaction between Node.js, cryptographic certificate handling, and Revenue's PIT3 security requirements. The matter was investigated in depth to ensure the project aligned as closely as possible with real world software architecture.

9.3.1 Certificate Handling Limitations in Node.js

Revenue issues certificates in PKCS#12 (.p12) format containing:

- RSA private key
- Public certificate
- Associated password (Revenue generated)

Initial attempts were made to load and use the .p12 certificate directly in Node.js using the built-in crypto module, this resulted in repeated runtime errors:

- Property 'options.type' is invalid. Received 'pkcs12'
- DECODER routines::unsupported
- Cannot read properties of undefined (reading 'import')

These errors stemmed from the fact that Node.js does not natively support PKCS#12 parsing, nor does it expose the OS-level cryptographic providers required for Revenue issued certificates.

9.3.2 Password Handling Discrepancies

The solution SD Worx Developers implemented (C#) in production systems performed the following transformation on the certificate password:

1. Convert password to ASCII
2. Apply MD5 hash
3. Base64 encode the result

4. Use the transformed password to load the certificate

This logic was replicated for this project in Node.js, however, Node’s cryptographic stack does not support applying this password transformation within PKCS#12 loading routines.

9.3.3 OpenSSL Extraction Attempts

To work around Node limitations, OpenSSL was used to extract the private key and certificate from the .p12 file (SSL.com, 2020):

- openssl pkcs12 -in cert.p12 -nocerts -nodes -out private-key.pem
- openssl pkcs12 -in cert.p12 -clcerts -nokeys -out cert.pem

Despite multiple attempts using:

- Original ROS password
- MD5-hashed password
- Base64(MD5) password
- Legacy OpenSSL flags (-legacy)

All attempts failed with the error “Mac verify error: invalid password?”.

9.3.4 Architectural Misalignment Between Node.js and Revenue PIT3

Revenue’s PIT3 REST API specification assumes that cryptographic operations are performed using X.509 certificates and RSA private keys managed by secure system-level cryptographic providers. The specification requires signing operations using certificates installed in the operating system and protected by non-exportable private keys (Revenue, 2024a; Revenue, 2024b).

The PIT3 documentation further specifies that submissions must be digitally signed using RSA-SHA512 and that private keys should remain securely stored and protected within the host operating system’s certificate infrastructure (Revenue, 2024c).

Node.js, by design, provides a cross-platform cryptographic abstraction layer built on OpenSSL and does not natively integrate with the Windows Certificate Store or other OS-level key management providers (Node.js, n.d.). As a result, Node.js cannot directly access non-exportable private keys protected by the operating system, nor does it provide native support for interacting with hardware-backed certificate stores (OpenSSL Project, n.d.).

While Node.js supports loading PKCS#12 (.p12) certificate files via OpenSSL bindings, this requires exporting the private key into application space, which conflicts with Revenue’s security model that assumes system-managed, non-exportable keys (Node.js, n.d.; Revenue, 2024b).

This represents an architectural incompatibility rather than a coding defect. The Revenue PIT3 specification is designed around enterprise cryptographic frameworks (e.g., .NET, Java) that interface directly with operating system certificate providers. Node.js, being runtime-isolated and OpenSSL-dependent, does not meet these assumptions without introducing additional architectural components.

9.3.5 Decision to Introduce a Minimal .NET Signing Microservice

Given the above constraints, a design decision was made to introduce a minimal .NET signing microservice, responsible for:

- Loading the Revenue issued certificate using the Windows Cryptographic API
- Signing canonical request strings using RSA-SHA512
- Returning signatures to the Node.js application

9.4 What I Would Do Differently

The implementation of a structured validation framework (e.g. Joi) would have been used to validate input. Greater consideration should have been given to the technical constraints of cryptographic signing within Node.js. The initial approach assumed PKCS#12 certificates could be handled natively, which proved to be a time consuming mistake, earlier evaluation would have reduced the time spent triaging and introducing the .NET microservice.

9.5 Future Improvements

Future development would focus on enhancing validation, security, error handling for partial submission failures, and introducing secure secrets management and finer grained reporting.

10. Use of Artificial Intelligence (Appendix C)

Artificial Intelligence (AI) was used as an assistive tool during this project.

Initial Research

AI was used in the initial research phase to aid with the understanding of ROS API requirements and Revenue Swagger documents. This included summarising technical documents into a more user-friendly manner and gaining an initial overview of technical specifications.

Code Comprehension and Data Flow Analysis

AI was used to interpret code provided by work colleagues. This included analysing SQL SPROC's regarding data flow and data compilation. AI assisted in breaking down complex data flows into understandable components.

Debugging and Problem Resolution

AI was used as a supplementary debugging tool to help identify potential causes of errors and for suggested troubleshooting approaches. This included diagnosing the digest header issue experienced during section 6.13.

Academic Writing and Documentation

AI was used to assist in the rewording of technical explanations into a more formal academic tone, while maintaining the original content. All final written material was composed by myself, ensuring authorship.

Ethical Considerations

I believe that the use of AI in this project adhered to academic guidelines. AI was used to assist learning and not to replace it. Independent learning and problem solving remained the primary aim of this project.

11. Conclusion

This project successfully demonstrated the design and implementation of a POC system for managing and submitting ERR data in line with Revenue specifications. The solution integrates frontend, backend, database, and external API components to model the full submission lifecycle, including payload generation, cryptographic signing, and ROS interaction.

References

European Union (2016) *Regulation (EU) 2016/679 (General Data Protection Regulation)*. Available at: <https://eur-lex.europa.eu/eli/reg/2016/679/oj> (Accessed: 15 January 2026).

Finance Act 2022 (Ireland), Section 897C. Available at: <https://www.irishstatutebook.ie/eli/2022/act/44/enacted/en/html> (Accessed: 15 January 2026).

Fielding, R. et al. (2014) *RFC 7231: Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content*. IETF. Available at: <https://datatracker.ietf.org/doc/html/rfc7231> (Accessed: 03 March 2026).

IETF (2013) *RFC 3230: Instance Digests in HTTP*. Available at: <https://datatracker.ietf.org/doc/html/rfc3230> (Accessed: 04 March 2026).

IETF (2015) *RFC 7616: HTTP Digest Access Authentication*. Available at: <https://datatracker.ietf.org/doc/html/rfc7616> (Accessed: 03 March 2026).

Klyne, G. and Newman, C. (2002) *RFC 3339: Date and Time on the Internet — Timestamps*. IETF. Available at: <https://www.rfc-editor.org/rfc/rfc3339> (Accessed: 05 March 2026).

Microsoft (n.d.) *Windows Certificate Store Architecture*. Available at: <https://learn.microsoft.com/en-us/windows/win32/seccrypto/cryptography-portal> (Accessed: 03 March 2026).

MongoDB (n.d.) *MongoDB Manual*. Available at: <https://www.mongodb.com/docs/> (Accessed: 14 January 2026).

Node.js (n.d.) *Node.js Documentation*. Available at: <https://nodejs.org/docs/latest/api/> (Accessed: 14 January 2026).

Node.js (n.d.) *Crypto Module Documentation*. Available at: <https://nodejs.org/api/crypto.html> (Accessed: 03 March 2026).

OpenSSL Project (n.d.) *OpenSSL Documentation*. Available at: <https://www.openssl.org/docs/> (Accessed: 03 March 2026).

OWASP (2021) *OWASP Top Ten Web Application Security Risks*. Available at: <https://owasp.org/www-project-top-ten/> (Accessed: 01 March 2026).

Revenue Commissioners (2024a) *PAYE Employers – Enhanced Reporting Requirements (ERR) Overview*. Available at: <https://www.revenue.ie/en/employing-people/becoming-an-employer-and-ongoing-obligations/err-jan-2024/index.aspx> (Accessed: 15 January 2026).

Revenue Commissioners (2024b) *PAYE Employers REST API (PIT3) Documentation*. Available at: <https://revenue-ie.github.io/payee-employers-documentation/PIT3/rest/payee-employers-rest-api-pit3.html> (Accessed: 06 March 2026).

Revenue Commissioners (2024c) *Enhanced Reporting Schema – PIT3 (XSD)*. Available at: https://revenue-ie.github.io/payee-employers-documentation/PIT3/soap/v1/enhanced_reporting/enhanced_reporting-schema-pit3.xsd (Accessed: 05 March 2026).

SSL.com (2020) *How to Export Certificates and Private Key from a PKCS#12 File with OpenSSL*. Available at: <https://www.ssl.com/how-to/export-certificates-private-key-from-pkcs12-file-with-openssl/> (Accessed: 04 March 2026).

Svelte (n.d.) *Svelte Documentation*. Available at: <https://svelte.dev/docs> (Accessed: 20 January 2026).

SD Worx colleagues (2026) *Internal discussions on ROS handshake implementation* (Accessed: 22 February 2026).

OpenAI (2026) *ChatGPT – debugging and development support tool*. Available at: <https://chat.openai.com> (Accessed: 07 March 2026) (see Appendix C).

Appendices

Appendix A:

System Architecture Diagram: High-Level

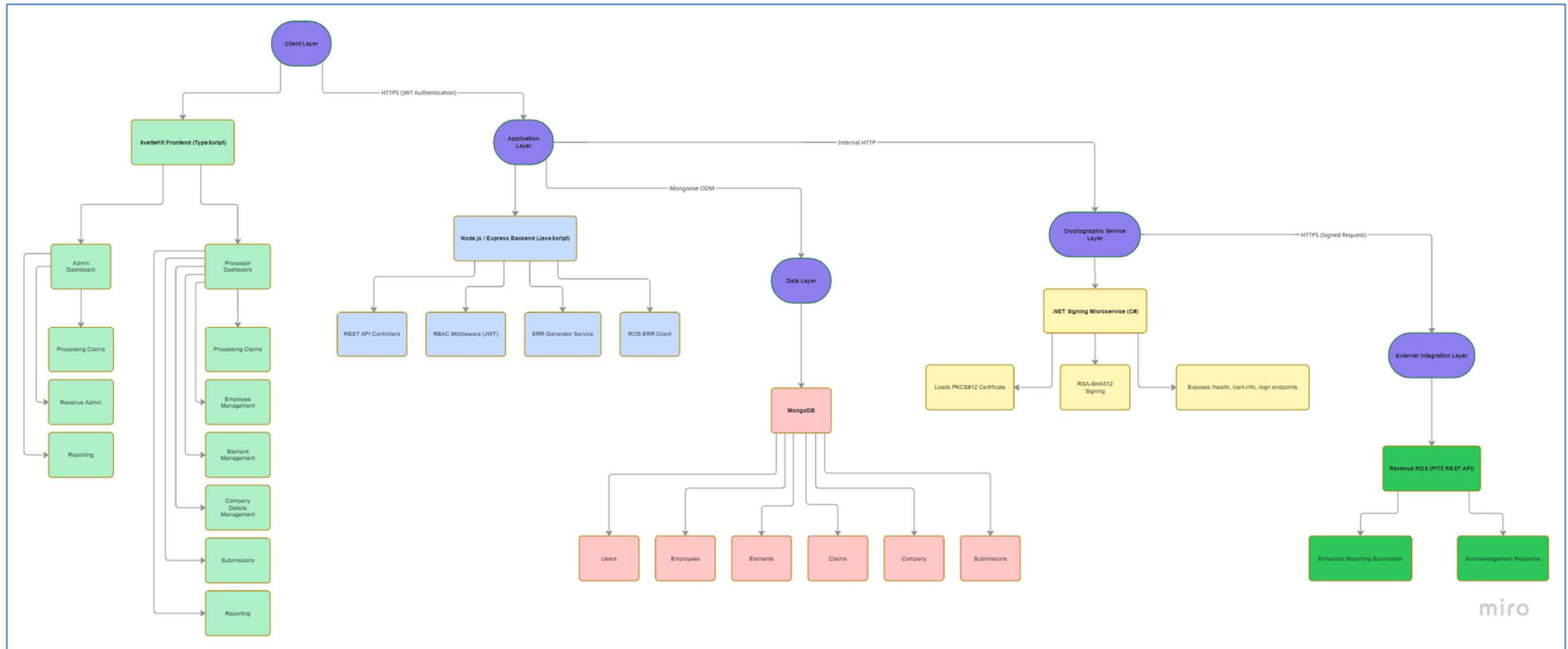


Figure 43: System Architecture Diagram: High-Level

System Architecture Diagram: Low-Levels

Frontend Client Layer

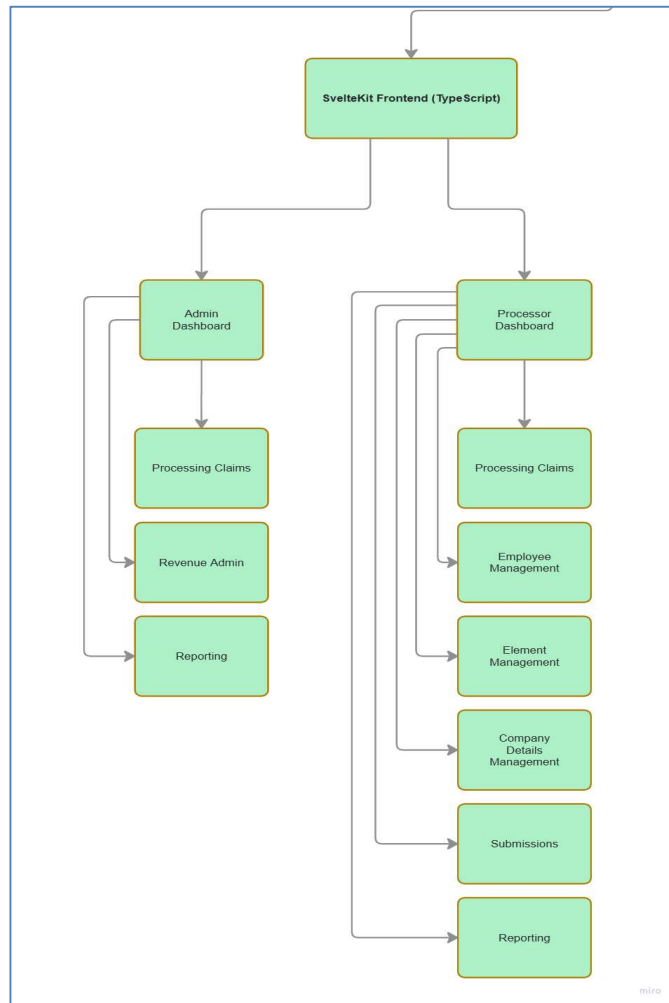


Figure 44: Architecture Diagram: Client Layer

Backend Application Layer

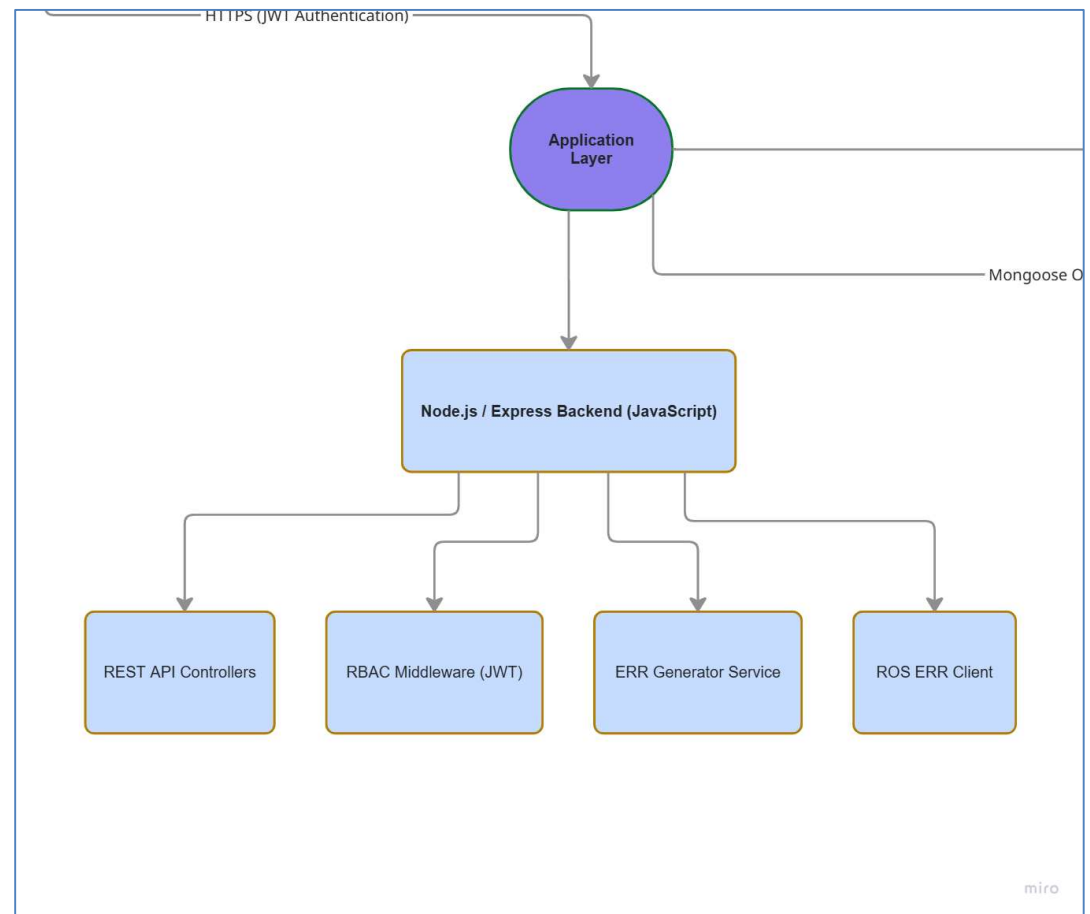


Figure 45: Architecture Diagram: Application Layer

System Architecture Diagram: Low-Levels

Data Layer

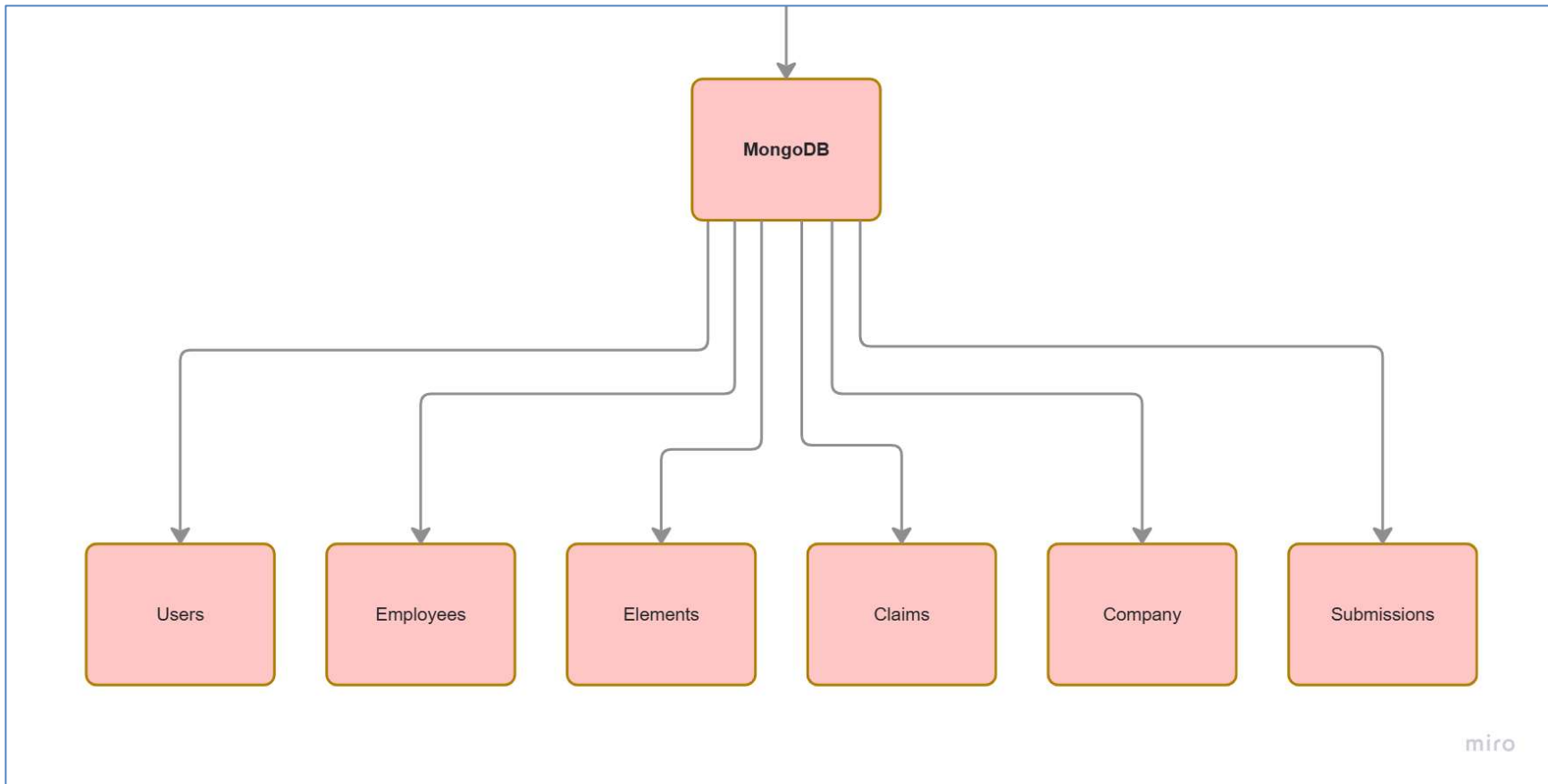


Figure 46: Architecture Diagram: Data Layer

System Architecture Diagram: Low-Levels

Cryptographic Service Layer

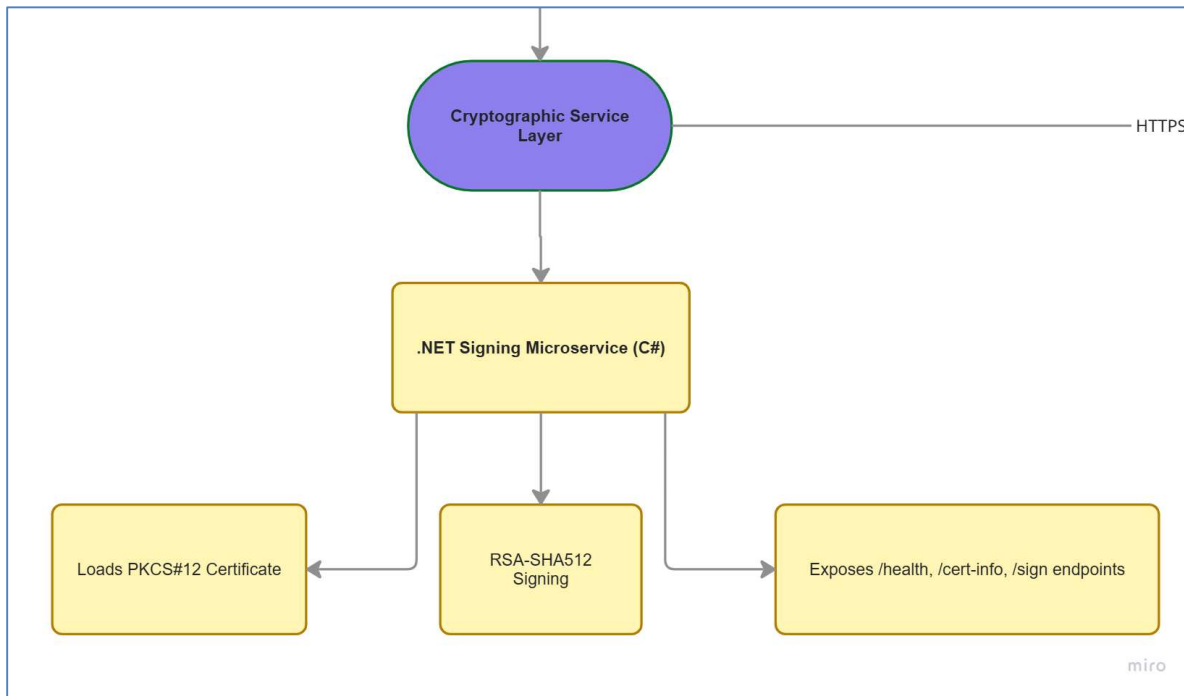


Figure 47: Architecture Diagram: Cryptographic Service Layer

External Integration Layer

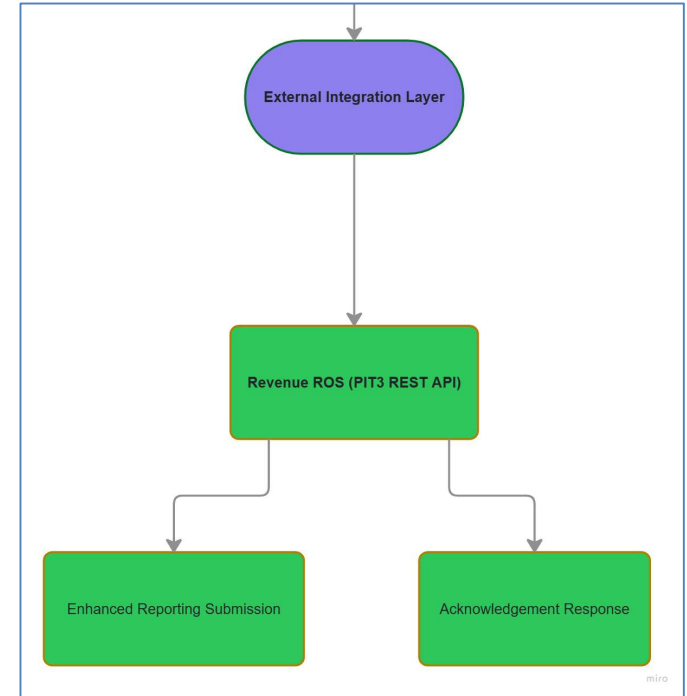


Figure 48: Architecture Diagram: ROS Integration Layer

Appendix B: File Tree

File Tree: ERR-Expenses-Web-App-with-ROS-API-Integration

```
├── Documentation
├── RevenueSigningService
├── Webpage
├── backend
│   ├── certs
│   ├── config
│   ├── controllers
│   ├── middleware
│   ├── models
│   ├── routes
│   ├── services
│   ├── tools
│   ├── utils
│   └── server.js
├── frontend
│   ├── src
│   │   ├── lib
│   │   │   ├── components
│   │   │   │   ├── icons
│   │   │   └── routes
│   │   ├── admin
│   │   ├── reports
│   │   ├── revenue
│   │   ├── home
│   │   ├── login
│   │   ├── processor
│   │   ├── company
│   │   ├── elements
│   │   ├── employees
│   │   ├── reports
│   │   └── submissions
│   └── static
└── README.md
```

Generated by FileTree Pro Extension

Appendix C: Use of AI, chat contents

1. Initial research on ROS API authentication:



ROS API ERR Authentication.pdf

2. Understanding of data compilation for submission provided by work:



ERR SPROC Data Flow.pdf

3. Debugging during section 6.13 ERR Submission Lifecycle and ROS API Integration:



AI Use_debugging.pdf

What authentication does revenue use for ROS API certificates for ERR submissions

Revenue's Online Service (ROS) uses digital certificates for authenticating API requests, including Enhanced Reporting Requirements (ERR) submissions. These certificates are essential for secure communication and must be properly configured to ensure successful interactions with ROS APIs. [api.sap.com](#) +6

Authentication Mechanism for ERR Submissions

1. Digital Certificate Requirement:

- All API requests to ROS must be digitally signed using a valid ROS digital certificate. This certificate must be obtained from ROS and is used to authenticate the identity of the requester. [Revenue](#) +2 [revenue-ie.githu...](#)

2. HTTP Signature Protocol:

- The digital signature is applied to the HTTP request using the HTTP Signatures specification. Revenue requires the use of the 'Signature' HTTP header method, where the signature is included in the HTTP headers of the request.

[revenue-ie.githu...](#) +1

3. Signature Components:

- The HTTP Signature includes the following components:
 - `keyId` : A Base64-encoded representation of the X.509 certificate.
 - `algorithm` : The algorithm used for signing, which must be `rsa-sha512`.
 - `headers` : A list of HTTP headers included in the signature.
 - `signature` : The actual digital signature, Base64-encoded.

4. Certificate Permissions:

- Employers' main ROS certificates are automatically granted ERR permissions. However, sub-user certificates require explicit assignment of ERR permissions through the ROS permissions screen. Without appropriate permissions, ERR submissions will be rejected. [revenue-ie.githu...](#) [Chartered Acco...](#) +5

5. Error Handling:

- If a request lacks a valid digital signature or the certificate does not have the necessary permissions, ROS will respond with HTTP status codes such as 401 (Unauthorized) or 403 (Forbidden). Detailed error messages will be provided in the response body to aid in troubleshooting.

Implementation Considerations

- **Certificate Management:**
 - Ensure that the digital certificate is stored securely and is accessible to the application making the API requests.
 - Regularly monitor the certificate's expiration date and renew it as necessary to prevent authentication failures.
- **Request Signing:**
 - Implement the HTTP Signature protocol correctly, ensuring that all required headers are included and the signature is computed accurately. revenue-ie.github... +1
- **Testing and Validation:**
 - Utilize Revenue's Public Interface Testing (PIT) environment to test API integrations before deploying to the live environment. revenue-ie.github...

By adhering to these authentication requirements and best practices, you can ensure secure and successful ERR submissions to Revenue's ROS APIs.

         Sources

A bit more technical now; is it OAuth or??

Revenue's ROS API for ERR (Enhanced Reporting Requirements) **does not use OAuth**.

Technical Summary of Authentication Method

Authentication Type: HTTP Signature using X.509 Digital Certificates

- No OAuth 2.0 or token-based authentication is used.
- Instead, authentication is done using:
 - A client-side X.509 digital certificate issued by Revenue.
 - The certificate is used to **digitally sign each HTTP request** using the `Signature` HTTP header.

How It Works

1. Client possesses a private key + X.509 public certificate (from Revenue).
2. When making an API request:
 - The client hashes specific HTTP headers (e.g., `(request-target)`, `date`, `digest`).
 - The hash is signed using the private key (RSA-SHA512).
 - The `Signature` header includes:
 - `keyId` : Base64-encoded public certificate (or fingerprint).
 - `algorithm` : `rsa-sha512`.
 - `headers` : List of headers included in the signature.

- `signature` : Base64-encoded signature value.

Example Signature header:

```
Signature: keyId="base64-cert" algorithm="rsa-sha512" headers="(request-target) d
</> http
```

3. Revenue's server verifies the signature using the certificate and checks if:

- The certificate is valid and trusted.
- The user has ERR submission permission.

Summary

Feature	Used?
OAuth2	No
Token-based Auth	No
X.509 Certificate Auth	Yes
HTTP Signature Header	Yes

Let me know if you want a code example or how to generate the signature.



please find the url for Irish Revenue ROS scheme
specifications for Enhanced Reporting Requirements

The official Irish Revenue documentation for ROS Enhanced Reporting Requirements (ERR) API specifications is available here:

Primary Specification (REST API – PIT3)

- <https://revenue-ie.github.io/payee-employers-documentation/PIT3/rest/payee-employers-rest-api-pit3.html> ↗

Supporting Documentation Hub

- <https://revenue-ie.github.io/payee-employers-documentation/> ↗

ERR Business / Reporting Guidance (non-technical)

- <https://www.revenue.ie/en/employing-people/becoming-an-employer-and-ongoing-obligations/enhanced-reporting-requirements/index.aspx> ↗
-

What Each Contains

- **PIT3 REST API page**
→ Full technical specification (endpoints, headers, authentication/signature requirements, handshake, submission flow)
 - **Documentation hub**
→ Navigation across versions (PIT3, PIT4), guides, and integration material
 - **Revenue ERR guidance page**
→ Legal/business rules (what must be reported, when, categories like travel, small benefits, remote working)
-

If you need, I can extract the exact sections that define:

- authentication + signature construction
- handshake sequence
- submission payload structure

and condense them into a clean technical summary.



please extract the exact sections that define:

authentication + signature construction
handshake sequence
submission payload structure

Below are the **exact technical sections (condensed and structured)** from the Revenue ROS PIT3 documentation relevant to your implementation.

1. Authentication + Signature Construction

Authentication Model

- Uses X.509 digital certificates issued by ROS

- Uses HTTP Signature scheme (not OAuth)
 - Signing algorithm: RSA-SHA512
-

Required Headers (included in signature)

Typically:

- (request-target)
 - host
 - date
 - digest *(for POST requests with body)*
-

Step-by-Step Signature Construction

1. Create Digest (for requests with body)

Digest = SHA-256 hash of request body → Base64 encoded



Format:

Digest: SHA-256=<base64_hash>

2. Build Canonical String

Example:

```
(request-target): post /paye-employers/v1/rest/handshake
host: softwaretest.ros.ie
date: Tue, 01 Mar 2026 10:00:00 GMT
digest: SHA-256=abc123...
```



- Header names must be **lowercase**
 - Order must match the `headers` field in Signature
-

3. Sign Canonical String

- Algorithm: RSA-SHA512
 - Use **private key** from ROS certificate
 - Output: Base64 encoded signature
-

4. Construct Signature Header

```
Signature:
keyId="<Base64 X.509 cert>",
algorithm="rsa-sha512",
headers="(request-target) host date digest",
signature="<Base64 signature>"
```

Key Points

- keyId = Base64 encoded certificate
- Signature is calculated **per request**
- Revenue validates:
 - Signature correctness
 - Certificate validity
 - Certificate permissions (ERR access)

2. Handshake Sequence

Endpoint

```
</> http
```

Query Parameters

- softwareUsed
- softwareVersion
- employerRegistrationNumber
- agentTain (*if applicable*)

Handshake Flow

Step 1 – Build Request

- Method: GET

- Include query parameters
- No request body → no digest required

Step 2 – Build Canonical String

```
(request-target): get /paye-employers/v1/rest/handshake?softwareUsed=...&so  
host: softwaretest.ros.ie  
date: Tue, 01 Mar 2026 10:00:00 GMT
```

Step 3 – Sign Request

- Use RSA-SHA512
- Include (request-target) host date in signature

Step 4 – Send Request with Headers

```
GET /handshake?... HTTP/1.1  
Host: softwaretest.ros.ie  
Date: ...  
...  
... http
```

Step 5 – Revenue Validation

Revenue checks:

- Signature integrity
- Certificate registration
- Matching employer / agent details
- Certificate permissions

Step 6 – Response

Success example:

```
r  
</> JSON
```



Failure example:

```
{  
  "validationErrors": [  
    {  
      "code": "1011",  
      "message": "Certificate does not have appropriate permissions"  
    }  
  ]  
}
```

3. Submission Payload Structure (ERR)

Endpoint

```
</> http
```



Headers Required

- Content-Type: application/json
- Date
- Digest
- Signature

High-Level JSON Structure

```
{  
  "employerRegistrationNumber": "1234567A",  
  "submissionDate": "2026-03-01",  
  "expenses": [  
    {  
      "employee": {
```

```
    "ppsNumber": "1234567A"
  },
  "paymentDate": "2026-03-01",
  "category": "TRAVEL_AND_SUBSISTENCE",
  "amount": 100.00,
  "description": "Travel reimbursement"
}
]
```

Key Fields

Employer Level

- employerRegistrationNumber
- submissionDate

Expense Entry

- employee
 - ppsNumber *(or alternative identifier)*
- paymentDate
- category
 - Examples:
 - REMOTE_WORKING_DAILY_ALLOWANCE
 - SMALL_BENEFITS_EXEMPTION
 - TRAVEL_AND_SUBSISTENCE
- amount
- description
- days *(required for remote working)*

Business Rules (from spec)

- Remote Working Allowance
 - Must include days
 - Amount derived from daily rate × days
 - Small Benefit / Travel
 - Use amount directly
 - Dates must be ISO 8601 (YYYY-MM-DD)
-

Submission Flow

1. Build JSON payload
 2. Generate Digest (SHA-256 of body)
 3. Build canonical string:
 - (request-target)
 - host
 - date
 - digest
 4. Sign with RSA-SHA512
 5. Send POST request
 6. Receive response (success or validation errors)
-

Typical Response

</> JSON



```
{  
  "submissionId": "ABC123",  
  "status": "ACCEPTED"  
}
```

Or validation errors if incorrect.

< 3/3 >      ...

Do you like this personality?



All_ERR_SPROC.docx
Document

Sample_Submission.json
File

PAYERRSUB And PAYERRSUBCTRL For ERI4 2025091N

Unable to display
visualization

ERR SPROC

I've attached the sample submission json file, the relevant tables and the ERR SPROC

Could you explain what is happening and the data flow along with all relevant tables and field names

Below is a structured explanation of:

1. Overall ERR data flow
2. Each stored procedure's role
3. All relevant tables and key field mappings
4. How this produces the JSON submission structure

Your sample JSON is referenced here:  Sample_Submission

1 High-Level ERR Data Flow

The ERR process has four logical phases:

Phase 1 – Configuration

SPROC_CREATE_PAYIEERRSETT

Defines:

- Which payroll elements qualify for ERR
- Their **Category** and optional **SubCategory**
- Whether they use **Days**, **Hourly**, **BIK**, or **Allowance**

This populates:

PAYIEERRSETT



This is the category mapping layer.

Phase 2 – Create Submission Control Record

SPROC_CREATE_ERR_SUBMISSION_CONTROL

Creates a header/control record in:

PAYERRSUBCTRL



This represents the submission “container”.

Key fields inserted:

Field	Meaning
ESUC_PAYROLL_RUN	Payroll run reference (e.g. ERI4-2025091N)
ESUC_SUBMISSION_NO	Submission sequence number
ESUC_REGISTRATION_NUMBER	Employer registration
ESUC_COMPANY	Company ID
ESUC_STATUS	Initially 'N' (Not sent)
ESUC_TAXYEAR	Tax year
ESUC_SPERIOD	Pay period
ESUC_RUN_TYPE	Normal/Bonus

This corresponds to the outer submission metadata in your JSON:

- enhancedReportingRunReference
 - submissionID
 - taxYear
 - employerRegistrationNumber
-

Phase 3 – Build Submission Line Items

SPROC_CREATE_ERR_SUBMISSION

This is the core logic.

It:

1. Reads payroll data
2. Aggregates relevant elements
3. Maps them to ERR categories
4. Detects changes vs previous submissions
5. Inserts line records into:

PAYERRSUB



Status is set to:

ESUB_STATUS = 'P' (Pending)



Each record becomes one JSON `expensesBenefits` item.

Phase 4 – Confirm Submission

SPROC_IE_CONFIRM_ERR_SUBMISSION

This:

- Changes ESUB_STATUS from 'P' → 'S'
- Marks previous replaced records as 'D'

Status transitions:

Code	Meaning
P	Pending
S	Submitted
F	Marked for deletion
D	Deleted

2 Core Tables Involved

1. PAYMCOMP (Company Master)

Source of:

Field	Purpose
MCOMP_TAXYEAR	Tax year
MCOMP_PAY_PERIOD_NUMBER	Period
MCOMP_BONUS_RUN	Run type
MCOMP_PAY_DATE	Payment date
MCOMP_REGISTRATION_NUMBER	Employer reg no
MCOMP_AGENT_TAIN	Agent TAIN
MCOMP_RUN_NO	Payroll run number

Used in both header and line generation.

2. PAYIEERRSETT (ERR Category Mapping)

Created by:

SPROC_CREATE_PAYIEERRSETT



Defines mapping such as:

IEERRS_ID	IEERRS_CATEGORY_OUT	IEERRS_SUBCATEGORY_OUT
R00	REMOTE_WORKING_DAILY_ALLO WANCE	NULL
B00	SMALL_BENEFITS_EXEMPTION	NULL
XTE	TRAVEL_AND_SUBSISTENCE	EMERGENCY_TRAVEL
XES	TRAVEL_AND_SUBSISTENCE	EATING_ON_SITE

These map directly to JSON:

```
"category": "TRAVEL_AND_SUBSISTENCE",
```

```
<> JSON
```



3. PAYGCELMs (Payroll Element → ERR Mapping)

Links payroll elements to ERR categories.

Fields:

Field	Purpose
GCELMs_ELEM_TYPE	A/H/B
GCELMs_ELEMENT_ID	Payroll element ID
GCELMs_IEERRS_ID	Links to PAYIEERRSETT

This determines which payroll elements are included in ERR.

4. Payroll Source Tables

These supply amounts and days.

PAYEMPL

Employee master

- EMPL_EMPLOYEE_ID
- EMPL_NAME
- EMPL_FIRST_NAME
- EMPL_RSI_NO (PPSN)
- EMPL_PERMANENT_RECID (UID)

PAYEMTAX

Employment tax info

- EMTAX_EMPLOYMENT_ID
- EMTAX_SMART_ID (Employer Reference)
- EMTAX_PMOD_EXCLUDED

PAYSLP

Payslip

- SLPR_PAID_THIS_PERIOD
- SLP_PAYMENT_DATE

PAYTIMS

Time/element transactions

- TIMS_ELEMENT_TYPE
- TIMS_ELEMENT_ID
- TIMS_VALUE (Amount)
- TIMS_HOURS_AMOUNT (Days)

The procedure:

```
SUM(TIMES_VALUE)
```

`</>` SQL



creates:

JSON Field	Source
amount	SUM(TIMES_VALUE)
numberOfDays	SUM(TIMES_HOURS_AMOUNT)
paymentDate	SLP_PAYMENT_DATE

5. PAYERRSUB (Line Items Table)

Final generated ERR lines.

Key fields:

Field	Maps to JSON
ESUB_EMPLOYEE_UID	Employee identifier
ESUB_PPSN	employeePpsn
ESUB_EMPLOYMENT_ID	employmentID
ESUB_EMPLOYER_REFERENCE	employerReference
ESUB_FIRST_NAME	firstName
ESUB_NAME	familyName
ESUB_CATEGORY	category

Field	Maps to JSON
ESUB_SUBCATEGORY	subCategory
ESUB_DAYS	numberOfDays
ESUB_AMOUNT	amount
ESUB_PAYMENT_DATE	paymentDate
ESUB_STATUS	P/S/F/D
ESUB_REPLACED_SUB	Previous record replaced

Each row becomes:

```
{
  "lineItemID": "...",
  "employeeID": {...},
  "category": "...",
  "subCategory": "...",
  "paymentDate": "...",
  "amount": "..."
}
```

Example from your sample:  Sample_Submission

6. PAYERRSUBCTRL (Submission Header)

Stores:

- One row per submission
- Links all PAYERRSUB rows via:
 - ESUB_PAYROLL_RUN
 - ESUB_SUBMISSION_NO

Change Detection Logic

This is critical.

When creating a new submission:

1. Previous submissions (status S or F) are loaded.
2. Records are matched on:
 - EMPUID
 - CATEGORY
 - SUBCATEGORY

If unmatched:

- Old ones marked F (for deletion)

If data changed:

- New record marked ISACHANGE = 'Y'
- Old record marked F

If unchanged:

- No new record inserted

Only changed rows are inserted into PAYERRSUB.

How JSON Is Produced

The JSON structure you provided  `Sample_Submission` is built from:

Header → PAYERRSUBCTRL

- enhancedReportingRunReference → ESUC_PAYROLL_RUN
- submissionID → derived
- employerRegistrationNumber → PAYMCOMP.MCOMP_REGISTRATION_NUMBER
- taxYear → ESUC_TAXYEAR

expensesBenefits[] → PAYERRSUB

Each row where:

`ESUB_STATUS = 'P'`



becomes one item.

Mapping:

JSON Field	Database Field
employmentID	ESUB_EMPLOYMENT_ID
employeePpsn	ESUB_PPSN

JSON Field	Database Field
employerReference	ESUB_EMPLOYER_REFERENCE
familyName	ESUB_NAME
firstName	ESUB_FIRST_NAME
category	ESUB_CATEGORY
subCategory	ESUB_SUBCATEGORY
paymentDate	ESUB_PAYMENT_DATE
numberOfDays	ESUB_DAYS
amount	ESUB_AMOUNT

5 End-to-End Flow Summary

PAYTIMS + PAYSLP + PAYEMPL + PAYEMTAX



Mapped via PAYGCELMs → PAYIEERRSETT



SPROC_CREATE_ERR_SUBMISSION



Inserted into PAYERRSUB (status P)



SPROC_IE_CONFIRM_ERR_SUBMISSION



Status changed to S



Exported to JSON



6 Key Architectural Insight

The system separates:

- Configuration (PAYIEERRSETT)
- Payroll data (PAYTIMS etc.)
- Submission line storage (PAYERRSUB)
- Submission header (PAYERRSUBCTRL)

- Submission lifecycle (status transitions)

It is essentially a **versioned delta submission engine**, not a simple export.

If you want next, I can:

- Draw the exact relational diagram
- Walk through one specific line (e.g. Remote Working Allowance) from payroll transaction → JSON
- Or explain the submission number generation logic in detail

Sources



please summarise the sequencing regarding the compilation of the header for API transmission;
<https://revenue-ie.github.io/payee-employers-documentation/PIT3/rest/payee-employers-rest-api-pit3.html>

Thought for 21s >

The header compilation sequence for Revenue API transmission is:

1. Identify the request details first

Establish the HTTP method, full path, and any query parameters. These are needed because the special `(request-target)` value is built from the lowercase HTTP method plus the request path and query string. This becomes one of the core inputs to the signed headers list. Revenue IE

2. Add the mandatory transport headers

Revenue requires `host` and `date` to be present for signing. If a normal `Date` header cannot be added programmatically, `x-date` may be used instead. The timestamp must be in GMT-compatible form, and Revenue accepts several standard date formats. Requests are only valid within a 90-minute window around the supplied timestamp. Revenue IE

3. For POST requests, build the body-dependent headers next

If the request is **POST**, calculate the `Digest` header from the payload:

- convert the payload to bytes,
- hash it with **SHA-512**,
- Base64-encode the result,
- place that value in the `Digest` header.

For POST, `digest` must also be included in the signed headers list. `Content-Type` is required as an HTTP header for POST, even though it does not have to be part of the signature string unless you choose to include it. Revenue IE

4. Decide which headers will be signed

Build the `headers` parameter as a lowercase, quoted, space-separated list in the exact order the values will later be concatenated. Revenue states that order is important. Common examples are:

- GET: `"(request-target) host date"`

- POST: "(request-target) host date digest"

Additional headers such as `content-type` , `content-length` , Or `x-http-method-override` may be included where relevant. Revenue IE

5. Construct the canonical signing string

Using the header names listed in the `headers` parameter, assemble the signing string in the same order. Each line is formatted as:

- (request-target): <method> <path-and-query>
- host: <host-value>
- date: <date-value>
- digest: <digest-value>

Each line is separated by a newline except the last. If a header named in the signed list is missing, it must be added to the HTTP request before constructing the signing string. Revenue IE

6. Create the digital signature

Sign the canonical string using the private key associated with the ROS certificate and the algorithm `rsa-sha512` . Then Base64-encode the resulting signature. Revenue IE

7. Compile the final Signature header

The `Signature` header is then assembled from four components:

- `keyId` = Base64-encoded X.509 certificate
- `algorithm` = `rsa-sha512`
- `headers` = the ordered signed-header list
- `signature` = the Base64-encoded digital signature

This final header is attached to the HTTP request before transmission. Revenue IE

8. Transmit the request only after header completion

In practice, the sequence is:

- build request path/query,
- add `host` and `date` / `x-date` ,

≡ ChatGPT ▾

↑ ...

- build the canonical signing string,
- generate the cryptographic signature,
- assemble the `Signature` header,
- send the request. Revenue IE

A simple way to think about it is:

request metadata → required headers → digest if POST → ordered signing string → RSA-SHA512 signature → final `Signature` header → transmit. Revenue IE

If you want, I can next turn this into a very short step-by-step developer checklist.

